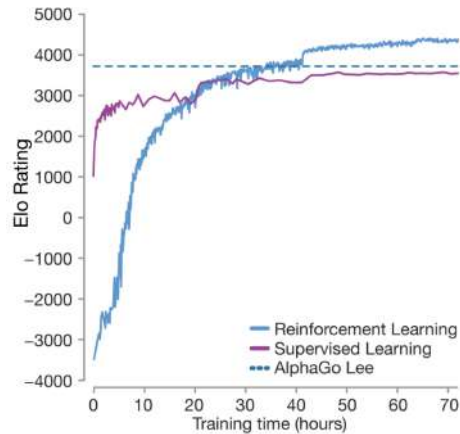


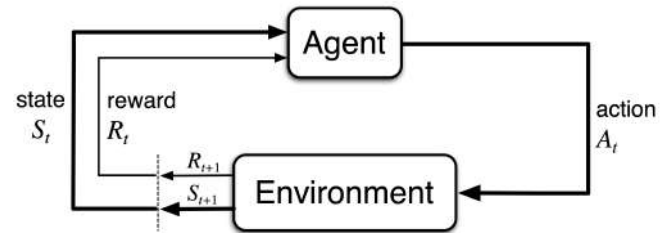
Reinforcement learning is very different from supervised learning. It is hard to master and can be challenging to implement even for experienced practitioners. But when it works, it is beautiful. This book aims to provide an understanding about why this might be the case, while also providing enough background to understand how and why RL is applied today.



The Little Book of Reinforcement Learning
A. Torres-Leguet

The Little Book of Reinforcement Learning

Alexandre Torres-Leguet



Contents

I Foundations	9
1 But what is RL?	11
1.1 Why is RL different?	11
1.2 The interaction loop	13
1.3 Observations and the Markov property	16
1.4 The training loop	19
1.5 Main challenges of RL	22
2 But how is RL done?	25
2.1 Learning a policy	25
2.2 Learning a model	26
2.3 Learning values	27
2.4 Common combinations	27
2.5 An example	31
Practical Notes	37

II Diving deeper	39
3 Value functions	41
3.1 The global view	41
3.2 Dynamic Programming	51
3.3 Monte Carlo Methods	54
3.4 SARSA	59
3.5 Q-learning	65
3.6 SARSA vs. Q-learning	68
3.7 Between MC and TD	70
3.8 Neural networks	76
4 Policy optimization methods	85
4.1 Setup	86
4.2 Policy gradient methods	87
4.2.1 Policy gradient theorem	
(weak form) - SPG	87
4.2.2 Policy gradient theorem	
(reward-to-go form) -	
REINFORCE	91
4.2.3 Baseline subtraction - VPG	92
4.2.4 Generalized Advantage Es-	
timation	96
4.2.5 SPG vs. REINFORCE	
vs. VPG	99
4.3 Trust region methods	101
4.3.1 Performance difference	
lemma	102
4.3.2 PPO	106

4.3.3 VPG vs. PPO	111
III RL at scale	115
5 RL $\times$ LLMs	117
5.1 Setup	120
5.2 From PPO to GRPO	121
5.3 Beyond GRPO	123
5.4 RL Infrastructure	128
5.5 Blurring the line between SFT and	
RL	132
6 AlphaGo Zero	135
6.1 The main idea	135
6.2 Improvement	136
6.3 Evaluation	139
6.4 Self-play	139
6.5 Discussion	141
Conclusion	143

Foreword

Reinforcement learning is very different from supervised learning. It is hard to master and can be challenging to implement even for experienced practitioners. But when it works, it is beautiful.

This book aims to provide an understanding of why this might be the case, while also giving enough background to understand how and why RL is applied today.

Its goal is not to be exhaustive, but rather minimalist and easy to read. For this reason, it follows the format of *The Little Book of Deep Learning* [Fleuret 2023]. Its tone, however, is closer to that of a blog post, as the book is built around a single narrative thread. Its structure broadly follows that of Sutton and Barto's *Reinforcement Learning: An Introduction* [Sutton et al. 2018], which remains the canonical reference on the subject.

This book is a reworked and expanded successor to a lecture series I gave in 2020, available at youtube.com/@alexandretl/courses.

Part I

Foundations

Chapter 1

But what is RL?

1.1 Why is RL different?

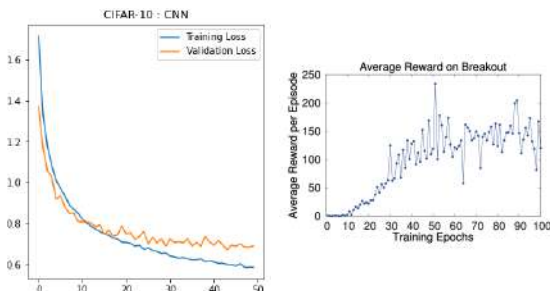
Reinforcement Learning (RL) is the study of agents that learn to act by trial and error: they interact with an environment, receive rewards, and adjust their behavior accordingly.

But when training a CNN to classify images from the CIFAR-10 dataset, isn't that already a decision problem? The model learns, during training, to decide how to classify a given image, and the learning signal tells it how well it is doing.

How is that different from a model learning to play Breakout, which updates its weights based

on the game score?

In both cases, the goal of training is to optimize some metric. So what's so different?



When classifying an image, the signal the model receives is the ground-truth label for that image. This signal is a direct target: it tells the model exactly what its output should have been. Optimizing against this signal is enough to learn to classify the image correctly.

When an agent plays a move in Breakout, however, the signal it receives is the game score. This score doesn't depend only on the action just taken: a good action may lead to a bad immediate score, and conversely, a bad action may lead to a good one. The reward is thus not a direct target as it does not tell the agent which action it should have played. This is the core difficulty of RL, and is often described as the *credit assignment problem*: the agent must figure out which of its past

actions deserve credit (or blame) for the rewards it eventually observes.

An **RL problem** involves an agent interacting with an environment, taking actions and receiving rewards as a consequence.

An **RL algorithm** provides a recipe to update the agent based on these rewards, so that it performs better in the future. In doing so, it tries to solve the credit assignment problem: it must determine which actions were responsible for the positive or negative rewards received, and reinforce or discourage them accordingly.

1.2 The interaction loop

As we saw, the interaction between the agent and the environment is at the core of RL. It is worth defining it precisely.

At each time step t , the agent observes the current situation of the environment, called the **state**, denoted S_t .

Based on this state, it chooses an **action** A_t .

Once this action is taken, the environment reacts: it transitions to a new state S_{t+1} and returns a **reward** R_{t+1} , which signals how well the agent is doing.

This creates the classical interaction loop:

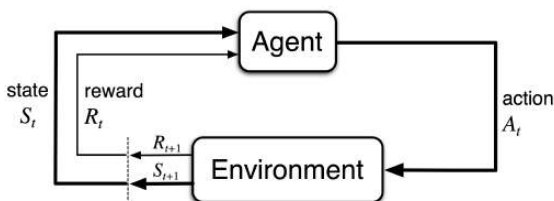


Figure 1.1: The RL interaction loop, as introduced by Sutton and Barto [Sutton et al. 2018].

The agent interacts with the environment repeatedly, producing a sequence called a **trajectory**:

$$S_0, A_0, R_1, S_1, A_1, R_2, S_2, A_2, R_3, \dots$$

When this trajectory reaches a terminal state, it is also called an **episode**. In a game, an episode may correspond to one full game; in robotics, to one trial. In *continuing tasks*, the interaction may continue without a clear end.

State S_t : what the agent knows about the current situation.

Action A_t : what the agent decides to do.

Reward R_{t+1} : the feedback signal received after taking action A_t in state S_t .

It is important to note that these definitions are abstract, and, for example, the reward can be 0 most of the time except at the end of the episode. Indeed, here are some examples:

Breakout

State: the current game screen, or features extracted from it (ball position, paddle position, brick layout).

Action: move left, move right, or stay still.

Reward: positive reward when hitting bricks or winning, negative reward when losing the ball.

Cart balancing

State: cart position, cart velocity, pole angle, and pole angular velocity.

Action: push the cart left or right.

Reward: +1 for every time step the pole remains balanced.

Robot navigation

State: robot position, distance to obstacles, and direction of the goal.

Action: move forward, turn left, turn right, or stop.

Reward: positive reward for reaching the goal, negative reward for collisions, small penalty for taking too long.

Recommender system

State: information about the current user, such as past clicks, watch history, or preferences.

Action: recommend one item among many possible choices.

Reward: positive reward if the user clicks, watches, or buys; little or no reward otherwise.

1.3 Observations and the Markov property

The definition of the interaction loop is universal: it imposes no limitation on the environments we may consider. To make the problem tractable, however, we usually assume one extra property of the environment: the **Markov property**.

It says that the current state S_t contains all the information needed to predict the evolution of the environment. In other words, no additional information from the past, like earlier states, is needed to predict what comes next. As the state fully describes the environment, the agent has all the information it needs to make a guess about the future and choose its action accordingly.

Importantly, the Markov property does **not** state that the future is determined by S_t . The next state and reward may still be random. It only says that whatever randomness there is depends on S_t alone, not on the full history.

Whether an environment is Markovian depends on how we define the state. In Breakout, for example, a single frame is not enough to tell where the ball is heading but a stack of a few additional past frames is. This violates the Markov property, as adding past information should not be necessary to predict the future.



Figure 1.2: Where is the ball heading?

A common fix is to modify the definition of the state: instead of a single frame, use a stack of the last 4 frames, which, taken together, give us information about the ball velocity and thus its future trajectory. In fact, one can always make an environment Markovian by including enough history in the state (although it may not be tractable).

That’s why sometimes we distinguish between the “state” (being the true and fully descriptive state of the environment) and the “observation” (a part of the state that the agent has access to). Often, we mean by “state” the “observation”, because we assume that the environment, as we define it, is Markovian.

The framework just described, a set of states $\mathcal{S}$, a set of actions $\mathcal{A}$, along with transition function $p(s' \mid s, a)$ and a reward function $r(s, a)$ that model the environment dynamics, is formally called a **Markov Decision Process (MDP)**. It is the mathematical object underlying most of RL.

This assumption has a direct consequence for the agent. Since S_t contains everything needed to predict the future, an optimal decision rule needs only depend on S_t . This decision rule is called the **policy**:

$$\pi(A_t \mid S_t).$$

The policy is what the RL algorithm ultimately learns.

1.4 The training loop

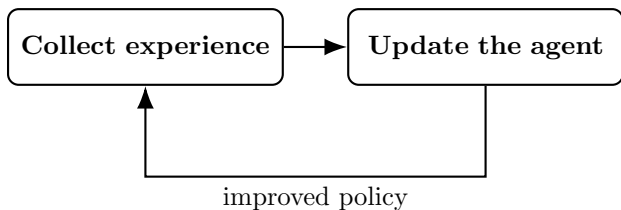
The interaction loop explains how the agent interacts with the environment and generates experience. The **training loop** explains how the agent improves from this experience.

A typical training loop looks like this:

1. the agent interacts with the environment,
2. it collects states, actions and rewards,

3. it uses this experience to update its policy,
4. it interacts again using its new policy.

So RL is usually an alternation between two phases:



Depending on the method, the update step may try to:

- increase the probability of actions that led to good outcomes,
- estimate how good a state or an action is,
- learn a model of the environment to better predict it,
- or combine several of these ideas.

So we are still taking a batch of data and using it to update a model, all of this inside a loop that goes on until we are satisfied with the performance of the model. Again, how is it so different from supervised learning? Certainly, as we saw, the update of the policy may not be the same as

in the supervised learning setting, but isn't the overall picture the same?

In a way, yes, but there is an important point that makes RL much more subtle: the data collected by the agent depends on the policy currently being followed. As the policy changes, the data distribution changes as well. A poor update can therefore lead the agent to visit very different states, collect worse experience, and receive fewer rewards, making learning collapse.

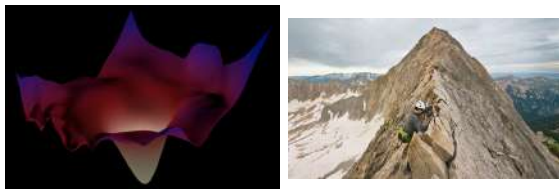


Figure 1.3: The loss landscape is much easier to navigate in supervised learning (left) than in RL (right).

In supervised learning, the data distribution is fixed. A bad update may hurt performance, but the model still learns from the same dataset, and the target it is trying to fit doesn't depend on its own predictions.

Interaction loop: how the agent and the environment produce experience.

Training loop: how this experience is used to improve the agent.

1.5 Main challenges of RL

- **Credit assignment problem**

A reward obtained at timestep 100 does not necessarily depend on the action taken at timestep 99. It may instead depend on an action taken much earlier, for example at timestep 50. This makes it difficult to determine which actions should be reinforced.

- **Optimization instability**

In reinforcement learning, the agent generates its own training data. As the policy changes, the data distribution changes as well. This makes training highly unstable: a poor update may lead the agent to collect worse trajectories, which in turn makes future learning harder. This is why many popular algorithms, such as Trust Region Policy Optimization (TRPO) or Proximal Policy Optimization (PPO), are built around the idea of taking careful policy updates.

- **Reward function**

Designing a good reward function is often one of the hardest parts of applying RL to a new problem. If the reward is too sparse, for example if it is only given at the end of the episode, a randomly initialized agent may never observe any useful signal. If it is too dense or poorly specified, the agent may learn to maximize the reward in a way that does not actually solve the intended task. This phenomenon is called *reward hacking*.

- **Exploration-exploitation dilemma**

The agent must choose between exploiting actions that already seem promising and exploring new actions that may lead to better outcomes. This trade-off is difficult because exploration may be costly in the short term, yet necessary to discover good strategies. During training, we want enough exploration to discover good strategies, but also enough exploitation to actually get a learning signal. During evaluation, we want to maximize exploitation to achieve the best performance.

Chapter 2

But how is RL done?

This chapter is focused on the taxonomy of RL algorithms. We will see that they differ by the objects they try to learn during the training loop.

2.1 Learning a policy

One of the objects an RL algorithm may learn is the policy itself, often denoted by π . It directly tries to learn which action to take. We usually talk about **policy-based** methods or **policy optimization**.

Given the previous batch of experience, update the policy to make it choose actions that lead to higher rewards.

If we are not directly learning a policy, we derive it from another object that we learn. This object usually conveys some kind of prediction about the future of the environment. Being in a given state, querying this object gives us some information about the different possible futures, allowing us to choose the best action. We derive the policy on the fly, without explicitly learning it.

2.2 Learning a model

One form this object can take is a model of the environment. This model is trained to predict how the environment evolves, typically its next state and reward, from the current state and action (thanks to the Markov property). This is called a **model-based** approach. So the direct goal here is not to maximize rewards, but to better predict the environment.

Given the previous batch of experience, update the environment model to better predict the future.

Once such a model has been learned, it can then be used to derive or improve a policy.

By contrast, **model-free** methods do not try to explicitly learn a model of the environment. Of course, these algorithms may still capture some model of the environment implicitly, but they do not learn an explicit predictive model that can be used to simulate or predict future states and rewards.

2.3 Learning values

The other form this object can take is a value function, often denoted by V or Q . The goal of such a function is to estimate how good a state is, or how good it is to take a given action in a given state, based on the rewards that are expected to be received in the future.

Given the previous batch of experience, update the value function to better estimate the value of the states and actions.

2.4 Common combinations

These objects each have a different nature and each tries to predict a different aspect of the environment. An RL algorithm may try to learn

several of these objects at the same time, and knowing which one it learns is one of the first reflexes to have when stumbling upon a new algorithm.

Rather than enumerating all combinations, we focus on the patterns that appear most often in practice.

Value-based, model-free methods

These methods do not try to learn a model of the environment. Instead, they learn a value function, most often an action-value function $Q(s, a)$, which estimates how good it is to take a given action a in a given state s . The policy is then derived implicitly and on the fly by choosing the action with the highest estimated value.

Deep Q-Network (DQN)

Policy-based, model-free methods

These methods also avoid learning an explicit model of the environment, but unlike value-based methods, they learn the policy directly. The goal is to directly increase the probability of actions that led to higher rewards in the past.

REINFORCE, Group Relative Policy Optimization (GRPO)

Actor-critic, model-free methods

Actor-critic methods combine both ideas. They learn a policy, called the actor, and a value function, called the critic. The actor is responsible for choosing actions, while the critic evaluates how good states or actions are, and helps guide and stabilize the learning of the actor. This combination is extremely common in modern RL, because it often provides a good compromise between performance and stability.

Trust Region Policy Optimization (TRPO), Proximal Policy Optimization (PPO)

Model-based methods

Model-based methods learn, or sometimes are given, a model of the environment. This model predicts how the environment evolves after an action is taken. The main advantage is that it allows planning: instead of acting immediately, the agent can use the model to imagine possible futures and choose actions more carefully.

Probabilistic Ensembles with Trajectory Sampling (PETS), AlphaGo Zero

Hybrid methods

Some algorithms combine several of the previous ideas at once. For example, an algorithm may learn a policy, a value function, and a model of the environment simultaneously.

MuZero, Dreamer

RL algorithms

Model-free

- **Value-based:** learn how good each situation is
- **Policy-based:** learn which action to take
- **Actor-critic:** learn both

Model-based

- **Planning with a model:** learn a model
- **Hybrid methods:** learn a model along with a policy and/or value function

The choice of which category to use for a given problem depends on multiple factors, such as:

- the environment: is it natural / easy to model? If so, model-based methods can be effective. Is it easier to estimate values, or to directly learn a policy?
- the amount of data available: if data is limited, use an algorithm that is able to re-use past experience.

2.5 An example

Let us look at a complete reinforcement learning project: learning to land a STARSHIP rocket. In this project, the goal is to control the final phase of descent and make the vehicle land upright on a landing pad. The environment was built in Unity and the training was done with the Unity ML-Agents toolkit, using PPO [Schulman, Wolski, et al. 2017] (which is a model-free, actor-critic RL algorithm). The agent receives observations describing the state of the rocket, such as its position, speed, and orientation, and must choose among 162 discrete actions combining engine on/off, engine gimballing, and RCS commands.



Figure 2.1: 3D rendering of the RL environment. The rocket is firing its engines to slow down and land on the pad.

This is a natural RL problem because it's easy to frame as a game. The **state** corresponds to the physical situation of the rocket at a given time: where it is, how fast it moves, and how it is oriented. It's a vector containing 11 real-valued numbers. The **actions** are the control decisions available to the agent, such as firing the engines, changing their direction, or using side thrusters. The **reward** is meant to capture the objective of the task: a successful landing. In other words, we do not tell the agent *how* to land. We only define what counts as success, and let it discover a strategy through trial and error.

This problem also helps illustrate the four main challenges of RL we discussed above:

- **Credit assignment problem** Trajectories in the environment typically span thousands of time steps, and the reward is only given at the end of the episode. A successful landing may depend on a sequence of thousands of actions, and it is not clear which ones were responsible for the success. The agent must discover which actions to reinforce based on the (very) long-term consequences they have on the final reward.
- **Optimization instability**

This is how the rewards look during training.

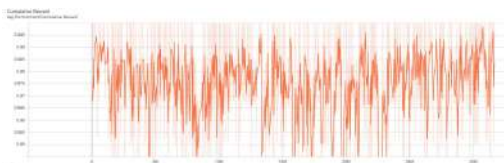


Figure 2.2: Reward curve during training.

Note that it is the only signal available during training for us to know if the agent is improving or not. This is very different from a smooth loss curve as seen in supervised learning.

- **Reward function** This has probably been the most difficult part of this project. Ideally, as said before, we would like to have a simple reward function that directly encodes the real goal of the task. That would imply giving a reward of +1 whenever the landing is successful, and 0 otherwise. However, this reward is extremely sparse: at the beginning of training, landing a rocket is almost impossible for a random policy (it starts 5000m in the air with a random orientation). Even given an enormous exploration budget, obtaining a single successful landing by chance is extremely unlikely, thus the agent never receives any learning signal.

One solution to this problem would be to give a denser reward, for example by giving a small positive reward for getting closer to the landing pad, a small negative reward for having too high a velocity, etc. But this is very delicate as it is prone to reward hacking: the agent may learn to maximize the reward in a way that does not actually lead to successful landings. For example, it may learn to hover at a certain height above the landing pad, which gives a good reward for being close, but never actually

land.

Another solution is to use curriculum learning: instead of starting from the full problem, we can start with an easier version of it, thereby making random exploration actually lead to some successful landings. For example, we can start by spawning the rocket much closer to the ground, or with a more favorable orientation. This way, the agent can experience success more easily, and then gradually learn to handle more difficult situations as training progresses.

The sparse reward function was finally chosen in combination with curriculum learning. It turned out not to be enough at the beginning of training, so imitation learning was also used briefly by providing demonstrations of successful behavior so that the agent could experience reward at least a few times. Once it had discovered trajectories leading to success, standard RL took over and started improving the policy further. Note that the imitation phase accounted for less than 0.001% of the total training. All the real progress was made thanks to RL on more and more difficult trajectories, spawning the spaceship straight from 500m at the beginning of the curriculum to 5000m

in the sky with random orientation at the hardest level. Note that this explains the non-monotonicity of the reward curve during training, as landings were made harder and harder as training progressed.

- **Exploration-exploitation dilemma** During training, the RL algorithm (PPO) ensures that the policy always has a bit of randomness, which allows for some exploration (i.e. try actions which are, according to the agent's knowledge, suboptimal). This is particularly important in this case because of curriculum learning: because the setup is changing during training, the agent must keep exploring and not get stuck in a local optimum that only works for the current curriculum level.

Learn more about this project at

github.com/alxndrTL/Landing-Starships.

Practical Notes

The companion repository for this book is available at:

github.com/alxndrTL/little-book-rl

It contains:

- One-file Python/PyTorch implementations of every algorithm (each linked in the body of the book)
- Supplementary material for proofs
- Notebooks to reproduce all figures

Part II

Diving deeper

Chapter 3

Value functions

This chapter focuses on detailing the different ways we can compute value functions. We will find that all methods, however different they may seem, follow a common pattern.

As we said in Chapter [2](#), such functions will be used downstream to derive a policy.

3.1 The global view

We are faced with the problem of estimating, for a given policy π , how good it is to be in a given state s . We call such an estimate a **value function**, denoted V^π , and define it for every

state s as the expected sum of future rewards the agent will collect, starting from s and acting according to π :

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s \right].$$

You need not worry about the fancy notation such as the expectation or the infinite sum:

- The expectation simply means: out of all the possible futures that can happen starting from s , **what is the average total reward I will get if I follow π ?**
- The infinite sum translates the fact that we care about the long-term reward, not just the immediate reward. We introduce a discount factor $\gamma \in [0, 1)$ to make sure that the sum converges even when the episode never ends. It has the side effect of making the agent care less about rewards that are far in the future.

The term inside the expectation is called the return and is denoted $G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$.

We will sometimes also need a finer-grained quantity, the **action-value function** Q^π , which estimates how good it is to take action a in state s

and follow π afterward:

$$Q^\pi(s, a) = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s, A_t = a \right].$$

The definition is the same as previously, except that we also condition on the first action a taken in state s . Both quantities give us a way to evaluate how good a state or action is under a given policy.

Example:

Consider a 4×4 grid where the agent starts in the top-left corner and wants to reach the bottom-right corner. At each step, it can move up, down, left, or right (actions that would take it off the grid leave it in place). It receives a reward of -1 on every step, and the episode ends when it reaches the goal. Maximizing reward is therefore equivalent to reaching the goal as fast as possible.

Consider the policy π that moves either right or down with equal probability. What does V^π look like for each of the states? You can try to compute it by hand for a few states (start with the states near the goal as they are easier).

-7.9	-6.9	-6.3	-6.0
-6.9	-5.5	-4.5	-4.0
-6.3	-4.5	-3.0	-2.0
-6.0	-4.0	-2.0	★

The values reflect the fact that the further away from the goal a state is, the more negative its value is. We thus have a measure of “goodness” of a state.

A different policy would yield very different values, and so this “goodness” is relative to the policy we study:

- the optimal policy π^* , which moves only toward the goal and avoids wasteful actions, would give $V^*(0, 0) = -6$, with the notation $V^* = V^{\pi^*}$.
- a policy π_1 that only moves right would give a value of $-\infty$ for all the states that are not in the last row.

Remark

In the previous example, the value function V^π took the form of an array, which was quite natural given the discrete nature of the state. In environments where the state is continuous, one natural solution is to discretize it and we are back to the previous case. Another solution, which is more common in modern RL, is to use a function approximator. We will develop this idea later in this book.

But how do we actually compute $V^\pi(s)$? If you tried the previous example by hand, you probably found that it involves some kind of recurrence. Let us look at one specific cell to make this explicit. Consider $V^\pi(3, 2)$, the cell just left of the goal, under the original policy π (right-or-down with equal probability). What can we say about it?

From this cell, π picks “right” or “down” with probability $1/2$ each:

- If it picks “right”, the agent reaches the goal: it collects a reward of -1 and the episode ends, so this branch gives a total future reward of -1 .
- If it picks “down”, the action is blocked (we are already on the bottom row): the agent

stays in place, collects -1 , and finds itself in the very same state $(3, 2)$, from which the same expected future awaits. This branch therefore gives a total future reward of $-1 + \gamma V^\pi(3, 2)$. The weight γ is there because the reward of -1 is collected immediately, while the future reward $V^\pi(3, 2)$ is collected one step later, and thus discounted by γ (we took $\gamma = 1$ in the example).

Putting both branches together:

$$V^\pi(3, 2) = \frac{1}{2}(-1) + \frac{1}{2}(-1 + \gamma V^\pi(3, 2)).$$

Solving for $V^\pi(3, 2)$ gives $V^\pi(3, 2) = -2$, which matches the value in the grid.

We see that the value of one state is expressed in terms of the values of its possible successors, whose values may themselves depend on the original state's value. This is not a one-shot computation.

Generalizing this reasoning gives the **Bellman equation** [Bellman 1961]:

$$V^\pi(s) = \sum_a \pi(a \mid s) \sum_{s'} p(s' \mid s, a) [r(s, a) + \gamma V^\pi(s')]$$

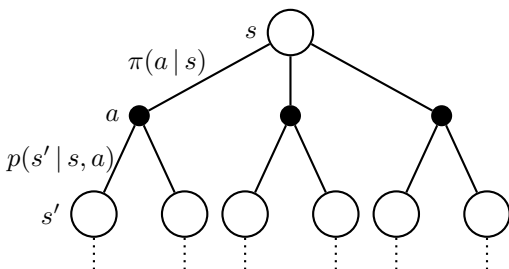
The outer sum averages over the actions that π may pick in s . The inner sum averages over the possible responses of the environment: each leads to a next state s' with a reward $r(s, a)$. This

future has a value of $V^\pi(s')$, discounted by γ (or put differently, it gives a discounted future reward of $\gamma V^\pi(s')$).

$\pi(a|s)$ is known because we are evaluating π , which is given. Neither $p(s'|s, a)$ nor $r(s, a)$ is known in general, except if we have a model of the environment. We keep them in the equation for now, as if we knew them.

We can visualize this situation graphically, where the future unfolds as a tree, each level of the tree corresponding to a time step in the future, and each branch corresponding to a possible action or new state.

This tree is what we call the **backup diagram**:



At the top is represented the current state s . From it, the agent can select among different actions, represented with black disks. Each branch is weighted by $\pi(a|s)$, that is, it has a probability $\pi(a|s)$ of happening. Next, from each action,

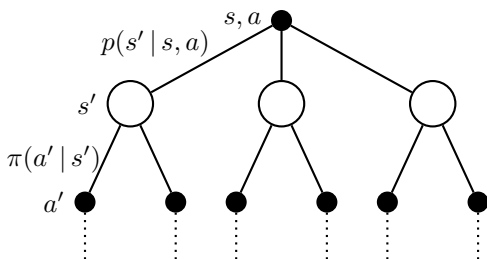
the environment can respond in different ways, leading to different next states. Each branch is weighted by $p(s'|s, a)$. This pattern then repeats indefinitely, until episode termination.

The value $V^\pi(s)$ is, by definition, the expected total reward collected when traversing this entire (possibly infinite) tree following π . So computing $V^\pi(s)$ amounts to traversing this tree, and summing the rewards collected along the way.

The same reasoning applies to the action-value function $Q^\pi(s, a)$. The only difference is that the action a is fixed (we are committed to taking it, no matter what π says to do in s), so the first level of branching disappears. This gives the **Bellman equation** for Q^π :

$$Q^\pi(s, a) = \sum_{s'} p(s' | s, a) \left[r(s, a) + \gamma \sum_{a'} \pi(a' | s') Q^\pi(s', a') \right]$$

The corresponding backup diagram starts with an action node rather than a state node:



The structure is the same as before, only shifted

by half a level: the tree now alternates “environment, then policy” instead of “policy, then environment”.

We have just generalized the computation of V^π from the small grid environment. Applied to that grid, the Bellman equation gives one equation per cell, which together form a system that fully characterizes V^π :

$$\left\{ \begin{array}{l} V^\pi(0,0) = \frac{1}{2}(-1 + V^\pi(0,1)) + \frac{1}{2}(-1 + V^\pi(1,0)) \\ \vdots \\ V^\pi(2,2) = \frac{1}{2}(-1 + V^\pi(2,3)) + \frac{1}{2}(-1 + V^\pi(3,2)) \\ \vdots \\ V^\pi(3,2) = \frac{1}{2}(-1 + V^\pi(3,3)) + \frac{1}{2}(-1 + V^\pi(3,2)) \\ \vdots \end{array} \right.$$

The 16 values displayed in the grid earlier are the unique solution of this system.

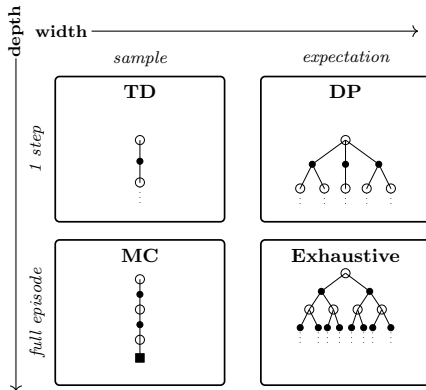
Note that in this case, we can write the whole equations because we know the dynamics of the environment (the $p(s'|s, a)$), which are in this case either 0 or 1. If we did not know the dynamics, we would not be able to expand the expectations of the Bellman equations, and would be stuck with:

$$\begin{aligned} V^\pi(s) &= \mathbb{E}_\pi[R_{t+1} + \gamma V^\pi(S_{t+1}) \mid S_t = s] \\ Q^\pi(s, a) &= \mathbb{E}_\pi[R_{t+1} + \gamma Q^\pi(S_{t+1}, A_{t+1}) \mid S_t = s, A_t = a] \end{aligned}$$

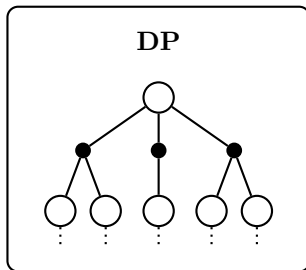
This reframing using the Bellman equation is what unifies all value-based methods. They all try to estimate V^π or Q^π by traversing, in some way, the backup diagram. The traversal is characterized by two questions:

- **How deep?** Do we expand the tree all the way down to the end of the episode, or do we stop early and replace the rest by a current estimate of V^π ?
- **How wide?** At each branching point, do we follow a single path by sampling, or do we expand all branches and average them with their probabilities? i.e. do we sample the expectation or compute it exactly?

Different combinations of answers to these two questions yield different families of algorithms. The remainder of this chapter is organized around this idea: we will see that Monte Carlo methods, Temporal-Difference learning, and Dynamic Programming, despite their very different appearances, are simply different ways of walking through the same tree.



3.2 Dynamic Programming



In this setting, we assume the dynamics of the environment, $p(s'|s, a)$ and $r(s, a)$, are known. The Bellman equation can then be applied directly, without sampling: at each state, we expand all branches of the backup diagram and weight them by their true probabilities, just as we did for the 4x4 grid example. This gives a system of $|\mathcal{S}|$

equations in $|\mathcal{S}|$ unknowns, which we can solve to find V^π . It is possible to solve it analytically, but this is rarely done in practice, as $|\mathcal{S}|$ can be large. We can instead iteratively apply the following Bellman operator:

$$L_\pi : \begin{cases} \mathbb{R}^{|\mathcal{S}|} & \rightarrow \mathbb{R}^{|\mathcal{S}|} \\ V & \mapsto R_\pi + \gamma P_\pi V \end{cases}$$

where $R_\pi + \gamma P_\pi V$ is the right-hand side of the Bellman equation written in vector form. Because L_π is a contraction mapping, the Banach fixed-point theorem guarantees that this iteration converges to V^π . This is the **policy evaluation** step.

Then, we would like to use V^π to find a better policy. The idea is quite simple: from state s , set $\pi'(s)$ to be the action that looks best according to V^π , i.e. create π_{k+1} such that:

$$\pi_{k+1}(s) = \arg \max_a \sum_{s'} p(s' | s, a) [r(s, a) + \gamma V^{\pi_k}(s')]$$

This is the **policy improvement** step. Alternating these two steps, evaluation and improvement, gives the **Policy Iteration** algorithm.

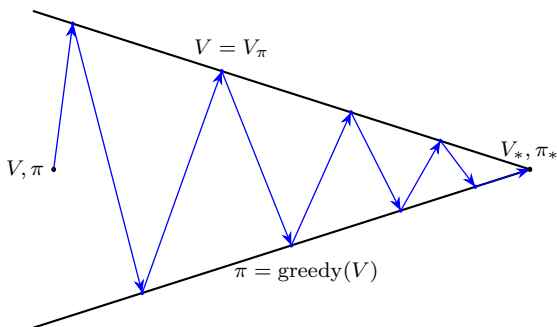
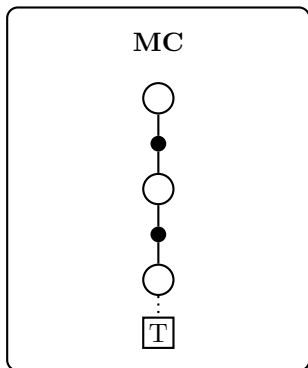


Figure 3.1: Policy iteration alternates evaluation and improvement, converging to (V^*, π^*) .

For further details on Dynamic Programming (formal proofs, pseudocode, and the Value Iteration algorithm), please refer to the supplementary material of this book.

Of course, the main limitation of this method is that it requires a perfect model of the environment, which is not available in most real-world problems. We will now see how to relax this assumption.

3.3 Monte Carlo Methods



We now drop the assumption of known dynamics. Without $p(s' \mid s, a)$, we can no longer compute the expectations in the Bellman equation. But an expectation can be estimated by sampling, so instead of directly computing:

$$V^\pi(s) = \mathbb{E}_\pi \left[G_t \mid S_t = s \right],$$

we will gather experience in the environment to compute an estimate of it. Concretely, we can take the average of the returns obtained after each visit to s :

$$V^\pi(s) \approx \frac{1}{N(s)} \sum_{t: S_t=s} G_t,$$

where $N(s) = |\{t : S_t = s\}|$ is the number of times s has been visited within the collected experience under π (across timesteps and episodes).

Regarding the backup diagram, instead of expanding all branches and going only one step deep, we sample a single path and follow it all the way to the end of the episode. We wait until the episode terminates and use the actual return to update our estimate of V^π . This is the **Monte Carlo evaluation** algorithm.

Its update rule is often written in an incremental form, meaning that when we visit S_t and observe the return G_t , we update $V(S_t)$ as follows:

$$V(S_t) \leftarrow V(S_t) + \alpha [G_t - V(S_t)],$$

where $\alpha > 0$ is a learning rate. With $\alpha = 1/N(S_t)$, this update is exactly equivalent to maintaining the running average as defined above.

The **Monte Carlo Control** algorithm is similar, but instead of updating V^π , we update Q^π and use it to improve the policy, similar to the improvement step in DP:

$$\forall s, \pi_{k+1}(s) = \arg \max_a Q^{\pi_k}(s, a).$$

Note that this is why we estimate Q^π rather than V^π : with V^π alone, comparing actions would require summing over next states with $p(s' | s, a)$, which we do not have access to here. Q^π absorbs this dependency. This is the reason why we introduced Q^π in the first place.

But here a new problem appears that did not exist in DP: in DP, the dynamics were known, so every action was implicitly “tried” through the expectation $\sum_{s',r} p(s',r \mid s,a)[\dots]$. Here, the only way to know the value of an action is to actually take it in the environment. If π_{k+1} becomes purely greedy, it will only ever pick the action that currently looks best, and the other actions will never be tried again. Their Q -values will never be updated, so we will never discover that one of them might in fact be better. The policy will get stuck on whatever initial estimates the agent happened to compute.

This is the **exploration-exploitation dilemma** we mentioned in Chapter [1](#), and it appears the moment the agent has to collect its own data. A fix is to never let the policy become fully greedy based on Q . Instead, we use an **ϵ -greedy** policy: with probability $1-\epsilon$, take the greedy action; with probability ϵ , take a uniformly random action. The improvement step becomes:

$$\pi_{k+1}(a \mid s) = \begin{cases} 1 - \epsilon + \frac{\epsilon}{|\mathcal{A}|} & \text{if } a = \arg \max_{a'} Q^{\pi_k}(s, a') \\ \frac{\epsilon}{|\mathcal{A}|} & \text{otherwise.} \end{cases}$$

The parameter $\epsilon \in [0, 1]$ controls the trade-off: a small ϵ exploits aggressively but risks missing better actions, while a large ϵ explores broadly but wastes time on actions currently considered

as suboptimal. In practice, ϵ is often set to a small value like 0.1, or decayed during training.

All the previous considerations give rise to the following algorithm:

Algorithm – MC Control

```
1: init  $Q(s, a)$ ,  $\pi$   $\epsilon$ -greedy with respect to  $Q$ 
2: for each episode do
3:   collect  $s_0, a_0, r_1, \dots, s_{T-1}, a_{T-1}, r_T \sim \pi$ 
4:    $G \leftarrow 0$ 
5:   for  $t = T-1, \dots, 0$  do
6:      $G \leftarrow \gamma G + r_{t+1}$ 
7:      $Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha[G - Q(s_t, a_t)]$ 
8:      $\pi(\cdot | s_t) \leftarrow \epsilon\text{-greedy}(Q(s_t, \cdot))$ 
9:   end for
10: end for
```

Code: [little-book-rl/algos/value_based/tabular.py](https://github.com/kevinzakka/little-book-rl/blob/master/algos/value_based/tabular.py)

There is an important remark to make about this formulation, regarding the interleaving of evaluation and improvement. In the pseudocode, the policy is updated right after each Q update (and so, equivalently, at the end of every episode). This does not respect the strict separation of evaluation and improvement steps as in DP, where we first evaluate the policy to convergence, then improve it.

In other words, we never wait for Q^{π_k} to be properly estimated before moving to π_{k+1} . As

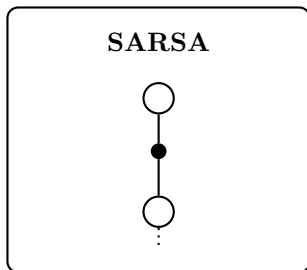
soon as a new return is observed, Q is updated and π is immediately made ϵ -greedy with respect to the new Q . The next episode is then collected under this new policy. Taking into account the new experience right away allows the policy to improve faster.

This also justifies why we use a constant learning rate α rather than $1/N(s)$. Since π keeps evolving, returns collected long ago were generated by a policy that no longer matches the current one. Averaging them with equal weight, as $1/N(s)$ would do, would bias the estimate toward the past. A constant α gives more weight to recent returns, which is desirable as the policy itself is a moving target.

The **Monte Carlo Control** approach has two drawbacks. First, it only works for episodic tasks: we need episodes to terminate before we can compute returns. Second, the variance of the returns can be very high, especially for long episodes, which makes the estimates noisy and thus slow to converge.

These drawbacks will be addressed in the next family of methods. What if, instead of waiting until the end of the episode to obtain a return, we used our current estimate of V^π to approximate the rest of the trajectory?

3.4 SARSA



SARSA is an instance of Temporal-Difference (TD) learning. TD is similar to Monte Carlo in that it relies on sampling a single path through the backup diagram, but it differs in that it does not wait until the end of the episode to update. Instead, it goes only one step deep and uses the current estimate of V^π to fill in the rest. This is referred to as **bootstrapping**.

Concretely, after observing a single transition $(S_t, A_t, R_{t+1}, S_{t+1})$, we update our estimate of $V(S_t)$ toward the value the Bellman equation predicts, using $V(S_{t+1})$ as a replacement for the rest of the future:

$$V(S_t) \leftarrow V(S_t) + \alpha \left[\underbrace{R_{t+1} + \gamma V(S_{t+1})}_{\text{target}} - V(S_t) \right]$$

The quantity inside the brackets is called the **TD error**: it measures the discrepancy between our current estimate $V(S_t)$ and a slightly better

one based on a real reward R_{t+1} followed by our estimate $V(S_{t+1})$ of what comes next. We nudge $V(S_t)$ to reduce this TD error.

Compared to MC, the target is now $R_{t+1} + \gamma V(S_{t+1})$ instead of the full return G_t . This addresses the two main drawbacks of MC: episodes no longer need to terminate, and the target has lower variance, since it depends on a single random reward and a single random transition, rather than on the entire future. However, the target is now **biased**: it depends on $V(S_{t+1})$, which is only an estimate. This makes TD learning particularly sensitive to the initialization of V (for example, if the initial values are far from the true values, the bias can take time to disappear).

The control counterpart, working on Q rather than V , gives rise to TD Control, also known as **SARSA**:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$$

The name SARSA comes from the quintuple $(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1})$ that the update consumes: state, action, reward, next state, next action.

Like in MC, we use an ϵ -greedy policy to ensure exploration during training:

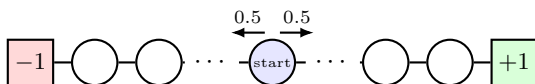
Algorithm – SARSA

```
1: Init  $Q(s, a)$ , set  $\pi$   $\epsilon$ -greedy with respect to  $Q$ 
2: for each episode do
3:   observe  $s_0$ , sample  $a_0 \sim \pi(\cdot|s_0)$ 
4:   for  $t = 0, 1, 2, \dots$  until  $s_t$  terminal do
5:     take action  $a_t$ , observe  $r_{t+1}, s_{t+1}$ 
6:     sample  $a_{t+1} \sim \pi(\cdot|s_{t+1})$ 
7:      $Q(s_t, a_t) \leftarrow$ 
        $Q(s_t, a_t) + \alpha[r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)]$ 
8:      $\pi(\cdot|s_t) \leftarrow \epsilon$ -greedy( $Q(s_t, \cdot)$ )
9:   end for
10: end for
```

Code: [little-book-rl/algos/value_based/tabular.py](https://github.com/kevinzakka/little-book-rl/blob/master/algorithms/value_based/tabular.py)

Example: MC vs. TD on random walk

To appreciate the differences between MC and TD, let us look at the **19-state random walk** (Sutton et al. (2018), Example 6.2 extended to 19 states). The agent starts in the middle of a chain of 19 states and moves left or right uniformly at random. Reaching the leftmost end gives a reward of -1 , reaching the rightmost end gives $+1$, all other transitions give 0 .



We compare MC and TD evaluation on this environment, both evaluating and following the same random policy.

Convergence speed. Using the RMS error between the estimated and the true value function (easily computed in this simple environment), we can track the convergence speed of both methods. We observe that TD converges much faster than MC.

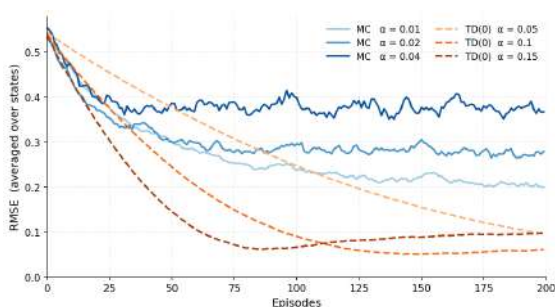


Figure 3.2: MC vs. TD on the 19-state random walk averaged over 200 runs, for different learning rates.

Variance of updates. MC sees returns which have much more variance than the TD targets, making its estimate noisier.

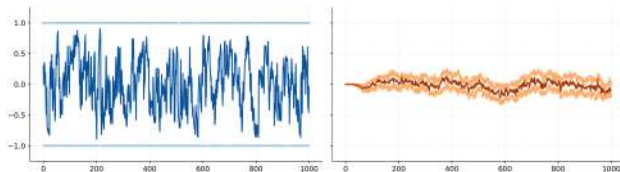


Figure 3.3: For MC (left, $\alpha = 0.02$) and TD (right, $\alpha = 0.1$), the value of the target (light) and G_t (dark) for state 10 through training.

Variance across runs. TD estimates have much lower inter-run variance than MC estimates. Indeed, MC targets are full Monte Carlo returns whose variance grows with episode length, while TD targets only depend on a single transition.

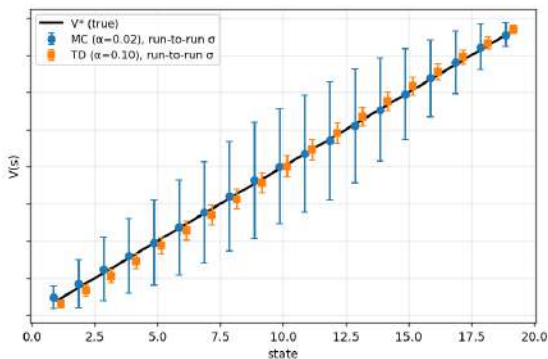


Figure 3.4: Estimate after 200 episodes.

Propagation of rewards. Finally, we can look at the estimated value function at different stages of training to see how rewards propagate back through the states of the chain. TD propagates rewards much more predictably than MC from the terminal states to the inner states. This is the result of TD doing bootstrapping, and thereby better exploits the Markov property of the environment. MC treats each state’s estimate independently and doesn’t leverage the structure of the value function across states. This hints at using TD for environments where we know the Markov property holds well, and using MC otherwise.

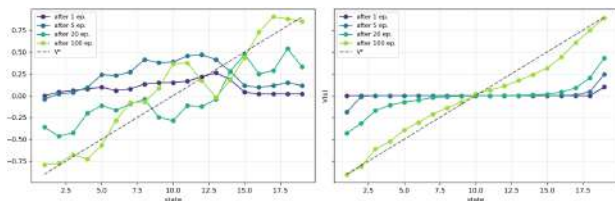
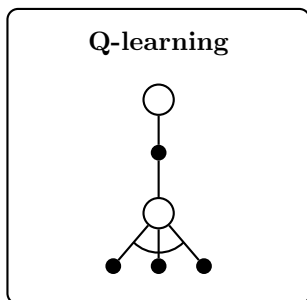


Figure 3.5: Value function found by MC (left, $\alpha = 0.02$) and TD (right, $\alpha = 0.1$). Lighter means later in training. This effectively shows how the rewards propagate in the value function from the terminal states to the inner ones.

3.5 Q-learning



Q-learning is another instance of TD, with one small but consequential change compared to SARSA. If we recall the SARSA update:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$$

The action A_{t+1} in the target is the action the agent *actually takes* at the next step, sampled from π . Since π is ϵ -greedy, A_{t+1} is the greedy action most of the time, but sometimes a random one due to exploration. So the target $R_{t+1} + \gamma Q(S_{t+1}, A_{t+1})$ reflects the value of π itself (including its exploration behavior). SARSA is evaluating its own behavior, which it then tries to improve.

This approach seems natural as we're trying to evaluate π which is exploratory. But we could also argue that exploration is added artificially for “mechanical” reasons and it is not intrinsic to

the problem nor ultimately something we want to optimize. We could try to ignore it for evaluation, while still using it to try actions and collect experience.

More generally, should we tie the target to the behavior we currently follow? i.e. from which policy should we sample a' to make the target?

SARSA, or **On-policy TD Control**, uses the same policy π for both acting and evaluating, that is why it is called an **on-policy** method.

Off-policy TD Control decouples the evaluation from the behavior. The general update still holds, but now the action that is actually taken in the environment is sampled from a **behavior policy** b :

$$A_t \sim b(\cdot \mid S_t),$$

while the action a' used to compute the target is sampled from the policy we want to evaluate, the **target policy** π :

$$a' \sim \pi(\cdot \mid S_{t+1}).$$

The behavior policy b is in charge of generating experience: it must explore. The target policy π is the one we actually evaluate and improve: it is the policy we ultimately care about.

One advantage of off-policy methods is that they can learn from experience generated by *any* policy,

not just the current one. This means we can reuse old experience collected under outdated policies. This will prove interesting when we move to deep RL, where data efficiency becomes important (this is why you have probably heard of Deep Q-learning but not Deep SARSA).

One natural choice is to set:

$$\pi = \text{greedy}(Q), \quad b = \epsilon\text{-greedy}(Q).$$

This is what **Q-learning** does, and its update is given by:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a') - Q(S_t, A_t)]$$

We can also view Q-learning as a way to directly estimate the optimal action-value function Q^* as its target is a sample of the Bellman optimality operator.

The rest of the algorithm is the same as SARSA so we omit the pseudocode here.

Code: [little-book-rl/algos/value_based/tabular.py](https://github.com/kevinzakka/little-book-rl/blob/master/algos/value_based/tabular.py)

Off-policy RL methods learn from transitions generated by a different policy than the one currently being trained. **Offline RL** methods learn from a fixed dataset of transitions, without any further interaction with the environment. Any offline RL method is hence off-policy.

3.6 SARSA vs. Q-learning

The classic **cliff walking** environment (Sutton et al. (2018), Example 6.6) clearly illustrates the different behaviors of on-policy and off-policy methods during training.

The agent navigates a small grid from a start cell to a goal cell. The bottom row, except for the two endpoints, is a cliff: stepping into it gives a reward of -100 and sends the agent back to the start. Every other step gives -1 . The agent thus wants to reach the goal as quickly as possible while avoiding the cliff.

The optimal path goes just above the edge of the cliff and takes 13 steps. We train both SARSA and Q-learning on this environment, with the same ϵ -greedy exploration ($\epsilon = 0.1$) and learning rate. We then track the sum of rewards collected per episode during training:



Figure 3.6: A visual representation of the cliff walking environment.

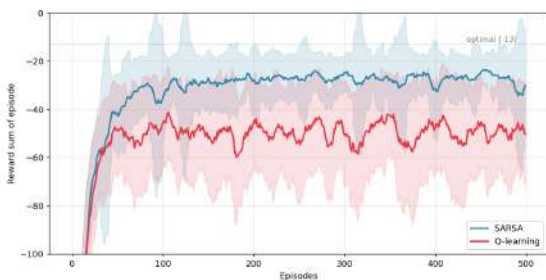


Figure 3.7: SARSA vs. Q-learning on cliff walking. Training rewards, averaged over 50 seeds. $\alpha = 0.5$, $\gamma = 1.0$, $\epsilon = 0.1$

SARSA performs much better during training, although the policy it learns is worse than the Q-learning one:

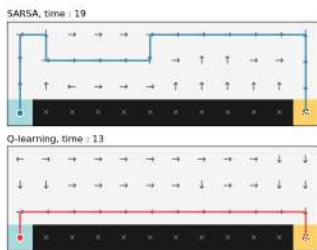


Figure 3.8: SARSA policy (top), Q-learning policy (bottom).

Q-learning learned the *optimal* policy because it didn't account for exploration during evaluation. All the times an action was chosen randomly and led to the cliff were not reflected in Q-learning's target. However, it was in SARSA's, so it learned a safer policy that avoids the cliff. If exploration is intrinsic to deployment (a real robot that may behave imperfectly), the safer behavior of SARSA may be preferable.

3.7 Between MC and TD

So far we have seen two extremes: MC, which uses the full return G_t , and TD, which uses a one-step bootstrap $R_{t+1} + \gamma V(S_{t+1})$. They sit at opposite ends of a spectrum, with opposite trade-offs: MC has high variance but no bias, TD has low variance but introduces bias through

bootstrapping. Can we interpolate between the two and get the best of both worlds?

n-step TD

Instead of bootstrapping after one step, we sum n real rewards and bootstrap on the n -th next state. After visiting S_t , we define the n -step return as:

$$G_t^{(n)} = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n V(S_{t+n}).$$

The update rule is the same as TD, with $G_t^{(n)}$ in place of the one-step target:

$$V(S_t) \leftarrow V(S_t) + \alpha [G_t^{(n)} - V(S_t)].$$

Setting $n = 1$ recovers TD. Taking n large enough to reach the terminal state recovers MC. Intermediate values of n trade variance against bias: larger n uses more real rewards (less bias from bootstrapping) but accumulates more randomness (more variance).

However, **a single n is rarely optimal throughout**. The best choice depends both on the state and on the progress of training:

- **Spatially**, states whose neighbors have accurate value estimates can use a small n , since the bootstrap $V(S_{t+n})$ is trustworthy, but states surrounded by poorly estimated

values need a larger n to look past the biased bootstrap and reach real reward.

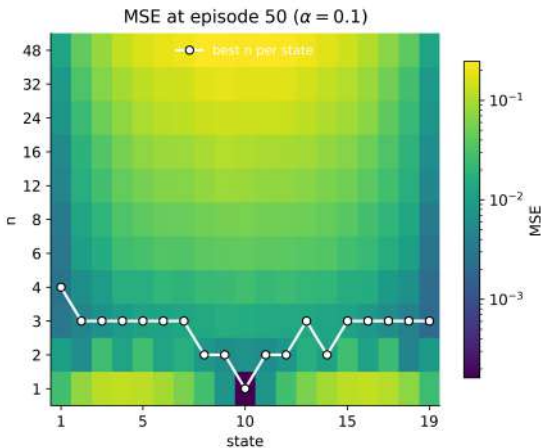


Figure 3.9: MSE of n -step TD evaluation for every state, for different values of n on the 19-state random walk. The optimal n varies by state. Near the terminal states (1 and 19), we can afford larger n 's since we are near the end. For the middle state, the target is very noisy as the rest of the episode is very long, so a smaller n is better.

- **Temporally**, V is wrong everywhere at the start, every bootstrap is biased, and a large n is preferable. But as V converges, the optimal n drifts down, since bias has been absorbed and variance becomes the

dominant concern.

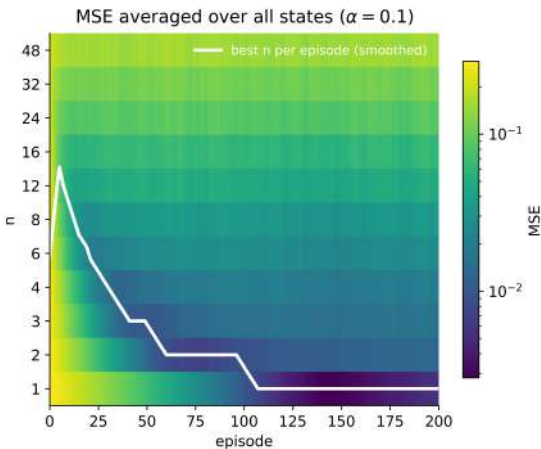
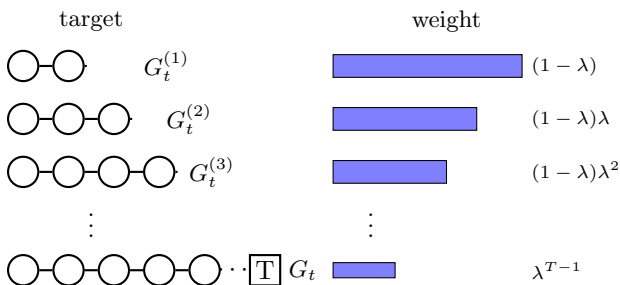


Figure 3.10: MSE of n -step TD evaluation at different times during training, for different values of n on the 19-state random walk. The optimal n in white decreases as training progresses.

TD(λ)

Hence, **TD**(λ) averages all n -step returns at once, weighted geometrically by $\lambda \in [0, 1]$:

$$G_t^\lambda = (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} G_t^{(n)}.$$



About the λ parameter:

- $\lambda = 0$ gives TD(0) (what we called TD up until now).
- low values of λ give more weight to short-term returns, but are smoother than a fixed n because they still average over all n .
- high values of λ give more weight to long-term returns.
- $\lambda = 1$ gives MC.

Code: [little-book-rl/algos/value_based/tabular.py](https://github.com/dennybritz/little-book-rl/blob/master/algos/value_based/tabular.py)

We can see how the n of n -step TD and the λ of TD(λ) affect the variance of the targets in the 19-state random walk:

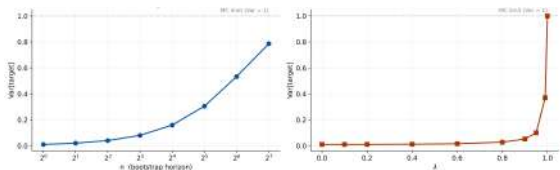


Figure 3.11: Target variance for state 10 for n -step TD (left) and TD(λ) (right), 19-state random walk.

Intuitively, TD(λ) can be seen as a way to “hedge our bets” in the bias-variance trade-off, which choosing a single n doesn’t address. Averaging over all n avoids the need to rely on a single value. And indeed, TD(λ) is Pareto-superior to n -step TD on the 19-state random walk:

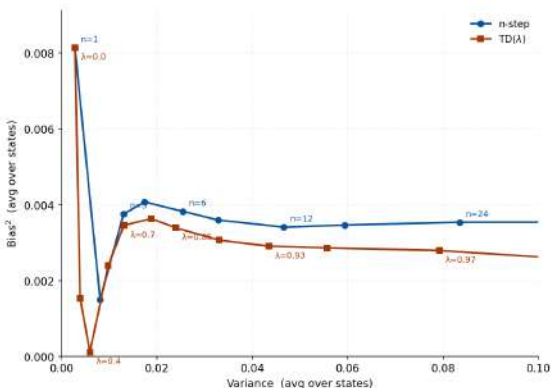


Figure 3.12: TD(λ) (red) Pareto-dominates n -step TD (blue) in this bias-variance plane ($\alpha = 0.1$, 19-state random-walk) (lower left is better).

This idea is extensively used in modern RL algorithms, notably in the **Generalized Advantage Estimation** (GAE, [Schulman, Moritz, et al. 2016]), which uses the same target as TD(λ):

$$A_t^{\text{GAE}(\lambda)} = G_t^\lambda - V(S_t) = \sum_{k=0}^{\infty} (\gamma\lambda)^k \delta_{t+k},$$

where $\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$ is the TD error.

3.8 Neural networks

So far in this chapter, we have assumed that states and actions are discrete and few enough to enumerate, making it possible to store Q in a table and update each (s, a) independently. This is problematic for two reasons:

- Many real-world problems have continuous state spaces (physics-based control tasks involving positions or velocities) or huge discrete ones (LLMs where the possible states are all possible token sequences). We could never visit and learn every cell during training.
- In the tabular case, each cell is learned in isolation, and the potential “closeness” between states is not exploited. However, we can expect the value function to be smooth

across states, i.e. we could generalize what we learn from one state to its neighbors.

The fix is **function approximation**: replace the table with a parametric function $Q_\theta(s, a)$ and learn θ . We typically use a neural network for Q_θ , and this is why we start talking about *Deep Reinforcement Learning*.

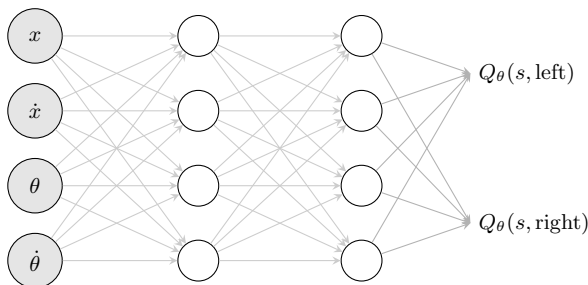


Figure 3.13: The Q-network on CartPole, where the goal is to balance a pole placed on a moving cart by moving the cart left or right. The input is the state of the environment (position, velocity, angle, angular velocity) and the output is the value of each action in that state.

In principle, any of the previous algorithms treated in the tabular case could be extended to the function approximation case. In practice, value-based deep RL is dominated by Q-learning-style methods because of their data efficiency that we talked about previously.

That is why every Deep Q-learning implementation you will encounter will have some kind of replay buffer $\mathcal{D}$ implemented, to store past transitions that are still valid to use for training the current policy.

Recall the Q-learning update from Section 3.5:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [y - Q(S_t, A_t)]$$

with $y = R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a')$.

This can be seen as a step toward the TD target y . With a parametric Q_θ , the natural analog is to minimize the squared distance to that target by SGD:

$$\mathcal{L}(\theta) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[(y - Q_\theta(s, a))^2 \right]$$

with $y = r + \gamma \max_{a'} Q_\theta(s', a')$.

Two complications arise from this:

- consecutive transitions in a trajectory are highly correlated, which violates the i.i.d. assumption of SGD. This is less problematic for off-policy methods as we can sample from any past transition.
- The target y depends on θ which we are updating.

These two complications make DQN different from standard supervised regression.

Historically, **Deep Q-learning** [Mnih et al. 2015] is the first algorithm to address these issues at scale and train a deep neural network to play Atari games at human level. The main innovation of DQN is the introduction of a target network, which is a separate copy Q_{θ^-} of the network that is used to compute targets:

$$y = r + \gamma \max_{a'} Q_{\theta^-}(s', a').$$

The target parameters θ^- are held fixed and only synced to θ every few thousand steps.

Remark

The architecture in the original paper is a small convolutional network reading a stack of four frames and the output layer has one head per action.

One practical limitation of DQN is that it only works with discrete action spaces, because the network has to produce one output per action and a max has to be applied on top.

The full algorithm:

Algorithm – Deep Q-learning (DQN)

```
1: Initialize replay buffer  $\mathcal{D}$ 
2: Initialize  $\theta$  at random; set  $\theta^- \leftarrow \theta$ 
3: for episode = 1, ...,  $M$  do
4:   Observe initial state  $s$ 
5:   repeat
6:      $a \leftarrow \epsilon$ -greedy( $Q_\theta(s, \cdot)$ );
7:     Execute  $a$ , observe  $r, s'$ 
8:     Store  $(s, a, r, s')$  in  $\mathcal{D}$ 
9:     Sample minibatch  $B$  from  $\mathcal{D}$ 
10:    for  $(s_j, a_j, r_j, s'_j) \in B$  do
11:       $y_j \leftarrow r_j + \gamma \max_{a'} Q_{\theta^-}(s'_j, a')$ 
12:    end for
13:    gradient step on loss  $\frac{1}{|B|} \sum_j (y_j - Q_\theta(s_j, a_j))^2$ 
14:     $s \leftarrow s'$ 
15:     $\theta^- \leftarrow \theta$  every  $C$  steps
16:  until  $s$  is terminal
17: end for
```

Code: [little-book-rl/algos/value_based/dqn.py](https://github.com/vwxyzjn/cleanrl/blob/main/algorithms/value_based/dqn.py)

Example: Atari Breakout

The CLEANRL [Huang et al. 2022] project provides a single-file, sub-300-line implementation of DQN that you can run on Atari games, in a very similar setup as in the original paper: github.com/vwxyzjn/cleanrl.

A core difficulty of RL training lies in the lack of reliable signal with which we can monitor the training. The reward curve is interesting to watch,

but it is a poor analogue of the supervised loss: it is noisy due to both environment stochasticity and ε -greedy exploration, and it is only indirectly related to what we actually optimize in this case, the TD error.

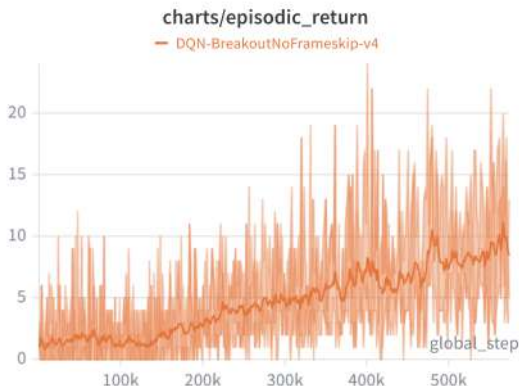


Figure 3.14: Return per episode during training. A few dozen minutes on a consumer laptop. Human level is around 30.

The TD loss, i.e. the regression loss of the Q-network, is not a good signal to track either. The regression target is a moving quantity which (we hope) changes over training as the policy improves and visits new states. The TD loss is therefore not the value of any fixed objective, and tracking it across training steps does not really make sense.

It may increase while training is healthy (a better policy is found and explores new regions of the state space), and it may decrease while training is stuck on a degenerate policy.

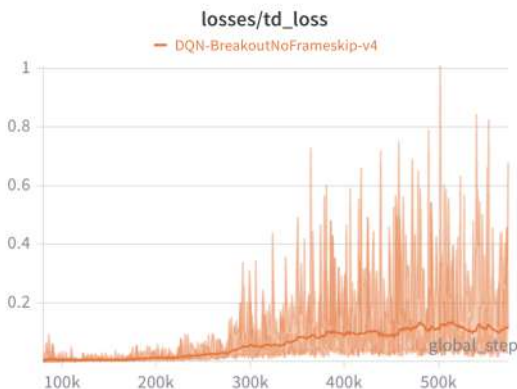


Figure 3.15: TD loss during training.

As opposed to supervised learning (e.g. language modeling), where the loss of a 1-trillion-parameter model can look as smooth as:

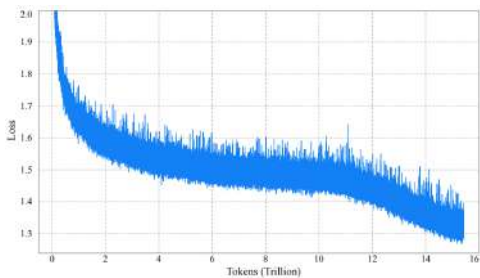


Figure 3.16: Kimi K2 training loss (1T total/32B active). It includes several improvements like MoE, Muon and synthetic data yet is still very stable.

Chapter 4

Policy optimization methods

In Chapter 3, we focused on learning value functions V or Q , and recovered a policy implicitly by acting greedily with respect to Q . The policy itself was never the object of optimization: we tried to make Q_θ satisfy a Bellman equation, and the policy was derived as $\arg \max_a Q_\theta(s, a)$.

In this chapter we take a different route: we parameterize the policy and directly learn its parameters. This is the domain of **policy optimization methods**.

4.1 Setup

We parametrize the policy as $\pi_\theta(a \mid s)$, typically a neural network mapping a state to a distribution over actions.

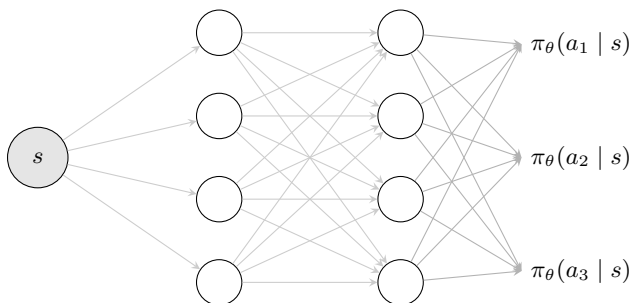


Figure 4.1: Policy network for a discrete action space with $|\mathcal{A}| = 3$. For a given state, the network outputs a distribution over actions.

We see right away how this can handle continuous action spaces, by outputting the parameters of a continuous distribution (e.g. mean and variance of a Gaussian).

We frame the problem of learning a policy that maximizes the expected return as the optimization of:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} [G(\tau)]$$

where $\tau = (S_0, A_0, R_1, S_1, A_1, R_2, \dots, S_T)$ is a

trajectory sampled by following π_θ in the environment, and $G(\tau) = \sum_{t=0}^{T-1} \gamma^t R_{t+1}$ is its discounted return.

The optimization problem is then

$$\theta^* = \arg \max_{\theta} J(\theta)$$

While in Chapter 3, the loss we minimized was only a proxy objective, here $J(\theta)$ is the quantity we care about (expected return) and we differentiate it directly.

Two families of algorithms tackle this problem differently:

- **policy gradient methods** compute $\nabla_{\theta} J(\theta)$ via the policy gradient theorem and apply gradient ascent. (**REINFORCE**)
- **trust region methods** build a surrogate objective that locally approximates J and optimize it within a trust region. (PPO)

4.2 Policy gradient methods

4.2.1 Policy gradient theorem (weak form) - SPG

The **policy gradient theorem** aims at giving a way to express $\nabla_{\theta} J(\theta)$ in a form that can be

estimated from samples. Its “added value” is that doing so is not trivial, as θ parameterizes the sampling distribution rather than the integrand, so differentiating under the integral does not directly yield an expectation.

There exist two forms of this theorem. The weaker form states that:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[G(\tau) \sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(A_t \mid S_t) \right].$$

We can in fact have a good intuition about this formula. For a trajectory τ that is sampled and from which we obtained the return $G(\tau)$:

- $\nabla_{\theta} \log \pi_{\theta}(A_t \mid S_t)$ is the direction in the space of parameters that makes action A_t more likely in state S_t .
- we sum these directions over all timesteps of the trajectory, which gives the direction of parameters that makes τ more likely (i.e. if we were to take a step toward that direction, τ would now be more likely). So this is a $\mathbb{R}^{|\theta|}$ vector, where $|\theta|$ is the number of parameters.
- the resulting vector is scaled by the trajectory return $G(\tau)$, a single scalar. If the trajectory was good ($G(\tau)$ large and positive), we step in the direction that makes

all of its actions more likely. If it was bad, we step in the opposite direction.

This formula can be approximated by sampling:

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N G(\tau_i) \sum_{t=0}^{T_i-1} \nabla_{\theta} \log \pi_{\theta}(a_t^{(i)} | s_t^{(i)}),$$

where $\tau_1, \dots, \tau_N$ are trajectories sampled by running π_{θ} in the environment.

This leads to the following algorithm, often referred to as **Simple Policy Gradient** (SPG, [Achiam 2018]):

Algorithm – SPG

```
1: Init policy parameters  $\theta$  at random
2: for iteration = 1, ...,  $M$  do
3:   Collect a batch of  $N$  trajectories  $\{\tau_i\}_{i=1}^N$  with  $\pi_{\theta}$ 
4:   for each trajectory  $\tau_i$  do
5:     Compute return  $G(\tau_i) = \sum_{t=0}^{T_i-1} \gamma^t r_{t+1}^{(i)}$ 
6:   end for
7:    $\hat{g} \leftarrow \frac{1}{N} \sum_{i=1}^N G(\tau_i) \sum_{t=0}^{T_i-1} \nabla_{\theta} \log \pi_{\theta}(a_t^{(i)} | s_t^{(i)})$ 
8:    $\theta \leftarrow \theta + \alpha \hat{g}$ 
9: end for
```

Code: [little-book-rl/algorithms/policy_based/spg.py](https://github.com/ikostrikov/little-book-rl/blob/master/algorithms/policy_based/spg.py)

A note on the “loss” in code.

In a typical PyTorch implementation of SPG, you will see something like:

```
loss = -(policy(obs).log_prob(act)*weight).mean()
```

where `obs` and `act` are the states and actions of the sampled trajectories, `weight` is $G(\tau)$ for SPG. The gradient will be computed by `torch.autograd`.

This quantity is called a “loss” by analogy with supervised learning, but it should not be considered a loss in the usual sense:

- its numerical value carries no information about how well the policy performs. While `grad(loss)` will indeed point in the opposite direction of $\nabla_{\theta}J(\theta)$, this does not imply that the value of `loss` is $J(\theta)$.
- Even as a gradient signal, it is only valid *at the parameters that were used to collect the data*. The trajectories (`obs`, `act`) stored in the batch were sampled from the old policy, and the policy gradient theorem requires the expectation to be taken under that same policy. So for the very first gradient step on this batch, the estimate is correct. But if we were to take a second gradient step on the same batch, θ would have moved and `grad(loss)` would no longer estimate $\nabla_{\theta}J(\theta)$. This makes policy gradient methods inherently *on-policy*, which is part of why they tend to be less data-efficient than the off-policy Q-learning methods of

the previous chapter (no replay buffer possible).

4.2.2 Policy gradient theorem (reward-to-go form) - REINFORCE

Under the weak form of the theorem, the same overall return $G(\tau)$ is used to weight all the actions taken in the trajectory via $G(\tau) \sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(A_t | S_t)$.

But it doesn't really make sense: for action A_t , the return $G(\tau)$ contains rewards $R_1, R_2, \dots, R_t$ that were collected *before* action A_t was taken. Those past rewards cannot have been caused by A_t . They have zero expectation and only add variance to the estimator.

The **reward-to-go form** of the policy gradient theorem gives a “better” expression:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^{T-1} G_t \nabla_{\theta} \log \pi_{\theta}(A_t | S_t) \right],$$

where G_t is the same G_t we used throughout Chapter 3 i.e. the cumulative discounted reward.

Both formulations are valid (they yield the same expectation) as the past rewards do not contribute to the expectation, only to its variance.

This makes the reward-to-go form less noisy.

This refinement gives us **REINFORCE** [Williams 1992], which is essentially SPG with $G(\tau)$ replaced by G_t :

Algorithm – REINFORCE

- 1: Init policy parameters θ at random
- 2: **for** iteration = $1, \dots, M$ **do**
- 3: Collect a batch of N trajectories $\{\tau_i\}_{i=1}^N$ with π_θ
- 4: **for** trajectory τ_i **do**
- 5: Compute returns $\forall t, G_t^{(i)} = \sum_{k=t}^{T_i-1} \gamma^{k-t} r_{k+1}^{(i)}$
- 6: **end for**
- 7: $\hat{g} = \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^{T_i-1} G_t^{(i)} \nabla_\theta \log \pi_\theta(a_t^{(i)} | s_t^{(i)})$
- 8: $\theta \leftarrow \theta + \alpha \hat{g}$
- 9: **end for**

Code: [little-book-rl/algos/policy_based/reinforce.py](https://github.com/dennybritz/little-book-rl/blob/master/chapter4/algos/policy_based/reinforce.py)

4.2.3 Baseline subtraction - VPG

While **REINFORCE** gives an estimate that has less variance than SPG, it is still noisy. A common variance reduction technique is to subtract a *baseline* $b(S_t)$ from the return G_t :

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{T-1} (G_t - b(S_t)) \nabla_\theta \log \pi_\theta(A_t | S_t) \right]$$

As long as $b(S_t)$ does not depend on A_t , this leaves the expectation unchanged, making the gradient

estimator unbiased for any choice of $b(S_t)$. But the variance depends on b , and a well-chosen baseline can make the estimator less noisy.

Indeed, consider $X = \pm 1$ with probability $1/2$ each, and let $Y = 100 \pm 1$. We want to estimate $\mu = \mathbb{E}[XY]$. The true value is

$$\mathbb{E}[XY] = \mathbb{E}[X]\mathbb{E}[Y] = 0.$$

The naive estimator $\hat{\mu} = XY$ takes values ± 99 or ± 101 and thus has variance:

$$\frac{1}{4}(99^2 + 101^2 + 99^2 + 101^2) = 10,001.$$

If we instead consider the modified estimator $\hat{\mu}_b = X(Y - b)$ with $b = 100$, it still has expectation zero, but now $\hat{\mu}_b = X(Y - 100) = \pm 1$, so its variance is just

$$\frac{1}{4}(1 + 1 + 1 + 1) = 1.$$

and indeed, in practice, optimizing the two estimators gives very different profiles:

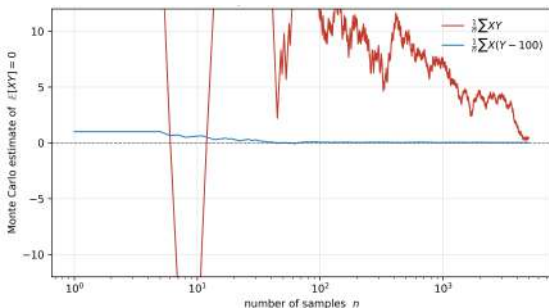


Figure 4.2: Convergence of the two estimators: $\hat{\mu} = XY$ (blue) and $\hat{\mu}_b = X(Y - 100)$ (orange).

The variable Y has a large mean (100) and a small fluctuation (± 1). Multiplying Y by a centered variable X amplifies the large mean into a ± 100 swing in the estimator, even though that mean carries no information about the quantity of interest (it gets killed in expectation by $\mathbb{E}[X] = 0$). Subtracting the mean of Y before multiplying removes the parasitic swing and keeps only the useful signal. A similar argument can be transposed to the policy gradient case, with X playing the role of $\nabla_{\theta} \log \pi_{\theta}(A_t | S_t)$ (centered) and Y the role of G_t (large mean, smaller action-dependent fluctuation).

A natural choice is $b(S_t) = V^{\pi_{\theta}}(S_t)$, the value function of the current policy: it represents the

mean return from S_t under π_θ . This gives:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{T-1} (G_t - V^{\pi_\theta}(S_t)) \nabla_\theta \log \pi_\theta(A_t | S_t) \right].$$

i.e. we increase the probability of an action proportionally to how much better than expected it turned out to be. We often introduce the **advantage** to designate this quantity:

$$A_t = G_t - V^{\pi_\theta}(S_t),$$

which measures how much *better than average* the action A_t turned out to be.

Thanks to Chapter [3](#), we know how to estimate V^{π_θ} . We therefore introduce a second neural network V_ϕ trained alongside π_θ to estimate the value function of the current policy. The policy network is often called the **actor**, while the value network is called the **critic** and the resulting family of algorithms is known as **actor-critic**.

We thus derive the **Vanilla Policy Gradient** (VPG) algorithm:

Algorithm – VPG

```
1: Init policy parameters  $\theta$  and critic parameters  $\phi$ 
2: for iteration = 1, ...,  $M$  do
3:   Collect a batch of  $N$  trajectories  $\{\tau_i\}_{i=1}^N$  with  $\pi_\theta$ 
4:   for each trajectory  $\tau_i$  do
5:     Compute  $\forall t \ G_t^{(i)} = \sum_{k=t}^{T_i-1} \gamma^{k-t} r_{k+1}^{(i)}$ 
6:     Compute  $\forall t \ A_t^{(i)} = G_t^{(i)} - V_\phi(s_t^{(i)})$ 
7:   end for
8:   Policy update:
9:    $\theta \leftarrow \theta + \alpha \frac{1}{N} \sum_i \sum_t A_t^{(i)} \nabla_\theta \log \pi_\theta(a_t^{(i)} | s_t^{(i)})$ 
10:  Critic update:
11:   $\phi \leftarrow \phi - \beta \nabla_\phi \frac{1}{N} \sum_i \sum_t \frac{1}{2} (G_t^{(i)} - V_\phi(s_t^{(i)}))^2$ 
12: end for
```

Code: [little-book-rl/algos/policy_based/vpg.py](https://github.com/vitchit/little-book-rl/blob/master/chapters/4.2.4/policy_based/vpg.py)

4.2.4 Generalized Advantage Estimation

So far, we estimated the advantage as $A_t = G_t - V(S_t)$. This is unbiased (assuming V is correct) but inherits the high variance of G_t , which is itself a single Monte Carlo sample of a long sum of stochastic rewards.

We can trade some of that variance for bias by using bootstrapped estimates of the return, exactly as we did in the transition from MC to TD methods in Chapter 3. The 1-step TD estimate

of the advantage is

$$A_t^{(1)} = R_{t+1} + \gamma V(S_{t+1}) - V(S_t) =: \delta_t,$$

which is just the TD error. The k -step estimate is

$$A_t^{(k)} = \sum_{l=0}^{k-1} \gamma^l R_{t+l+1} + \gamma^k V(S_{t+k}) - V(S_t),$$

and as k grows up to $T - t$, we recover the Monte Carlo advantage $G_t - V(S_t)$. Small k gives low variance and high bias (we trust V a lot) while large k gives high variance and low bias (we trust the actual rewards a lot).

As we saw in Section [3.7](#) with $\text{TD}(\lambda)$, rather than picking one k , the **Generalized Advantage Estimator** [Schulman, Moritz, et al. [2016](#)] takes an exponentially weighted average of all k -step estimates:

$$\begin{aligned} A_t^{\text{GAE}} &= (1 - \lambda) \left(A_t^{(1)} + \lambda A_t^{(2)} + \lambda^2 A_t^{(3)} + \dots \right) \\ &= \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l} \end{aligned}$$

with $\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$. Both $\gamma \in [0, 1]$ and $\lambda \in [0, 1]$ control the bias-variance trade-off:

- we can see γ as a way to reduce variance by down-weighting all future rewards. $\gamma < 1$ introduces bias.

- whereas $\lambda < 1$ introduces bias only when V is poorly estimated.

In practice, λ is usually set somewhere around 0.95, and GAE has become the default advantage estimator in modern policy gradient implementations. It is computed efficiently by a single backward pass through the trajectory using the recursion

$$A_t^{\text{GAE}} = \delta_t + \gamma \lambda A_{t+1}^{\text{GAE}},$$

with $A_T^{\text{GAE}} = 0$ at the end of an episode.

This yields the following actor update:

$$\theta \leftarrow \theta + \alpha \frac{1}{N} \sum_i \sum_t A_t^{\text{GAE}(i)} \nabla_{\theta} \log \pi_{\theta}(a_t^{(i)} | s_t^{(i)})$$

where the only change from VPG is the advantage estimator. For the critic, it is common to also switch to a λ -return target to reduce variance. Because of the relation $A_t^{\text{GAE}} = G_t^{\lambda} - V_{\phi}(S_t)$, the λ -return is available essentially for free and we can have the following critic update:

$$\phi \leftarrow \phi - \beta \nabla_{\phi} \frac{1}{N} \sum_i \sum_t \frac{1}{2} (\text{sg}(A_t^{\text{GAE}(i)} + V_{\phi}(s_t^{(i)})) - V_{\phi}(s_t^{(i)}))^2$$

where **sg** denotes the stop-gradient operator: the expression inside is computed with the current parameters but treated as a constant during differentiation.

4.2.5 SPG vs. REINFORCE vs. VPG

On CartPole, we compare the three algorithms:

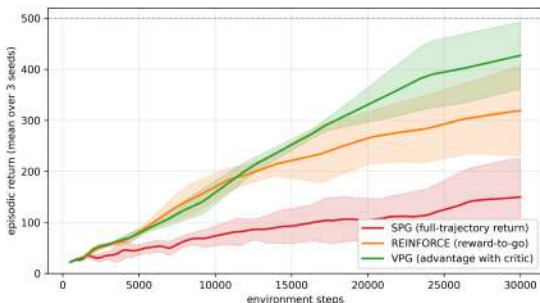


Figure 4.3: CartPole-v1: episodic return of SPG vs. **REINFORCE** vs. VPG through training.

As expected, $\text{SPG} < \mathbf{REINFORCE} < \text{VPG}$ in performance, and $\text{SPG} > \mathbf{REINFORCE} > \text{VPG}$ in relative inter-run variance.

Similarly to the previous XY example, we can plot the estimation of one of the components of the policy gradient. Subtracting the baseline indeed makes the gradient estimation smoother and leads to a graph very similar to the XY example:

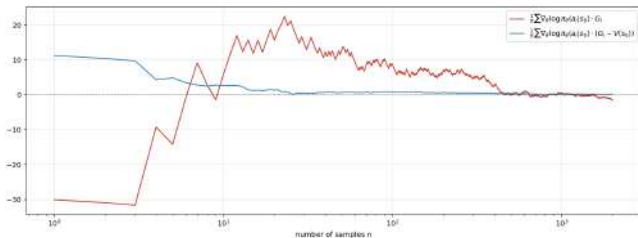


Figure 4.4: Convergence of two estimators of one component of the policy gradient at a fixed state. Red is no baseline (**REINFORCE**), blue is with baseline (VPG).

We can also directly evaluate the variance of the gradient estimates by stopping at a given iteration and sampling a large number of trajectories under the 4 different methods:

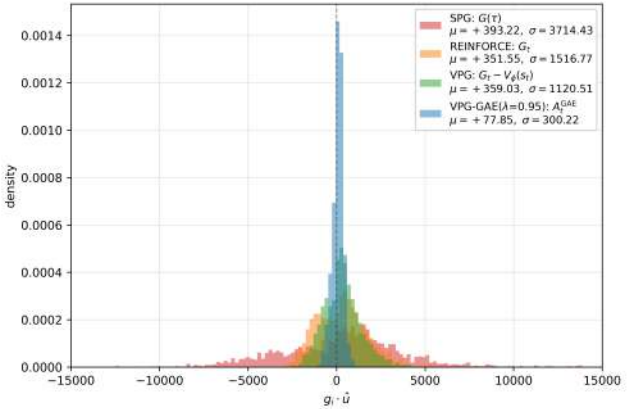


Figure 4.5: Distribution of gradient g_i for 2000 trajectories. We see a decreasing variance as we progress from SPG to VPG with GAE. Note the mean of each estimate: it is approximately the same for the first 3 methods because they are unbiased, but it is different for VPG with GAE because of the bias introduced by $\lambda < 1$.

4.3 Trust region methods

The previous policy gradient algorithms used the policy gradient theorem to get an estimator of $\nabla_\theta J(\theta)$ and then applied gradient ascent. Trust region methods take a different starting point: rather than differentiating J , we look at the difference $J(\theta) - J(\theta_{\text{old}})$ between a candidate new policy and the current one, and find that it admits

a form that can be approximated by a surrogate $L_{\theta_{\text{old}}}(\theta)$ estimable from rollouts of $\pi_{\theta_{\text{old}}}$ alone. The surrogate is only accurate in a neighborhood of θ_{old} , hence the name “trust region”: the region around θ_{old} in which we trust our surrogate.

This allows us to extract the most juice out of each batch of trajectories, since we now have an explicit objective we can optimize over several steps, as long as we stay in the trust region. On the contrary, we had to be more conservative with policy gradient methods: nothing in theory justified taking multiple gradient steps on the same batch, since the gradient estimator is only valid for the policy that produced the data.

4.3.1 Performance difference lemma

The starting point is an exact identity relating the returns of two policies. For any policies π_θ and $\pi_{\theta_{\text{old}}}$:

$$J(\theta) - J(\theta_{\text{old}}) = \frac{1}{1 - \gamma} \mathbb{E}_{s \sim d^{\pi_\theta}, a \sim \pi_\theta} [A^{\pi_{\theta_{\text{old}}}}(s, a)]$$

where $d^\pi(s)$ is the normalized state visitation distribution induced by π [Schulman, Levine, et al. 2015]. This formula states that the difference in performance between π_θ and $\pi_{\theta_{\text{old}}}$ is obtained by evaluating the new policy under the old critic.

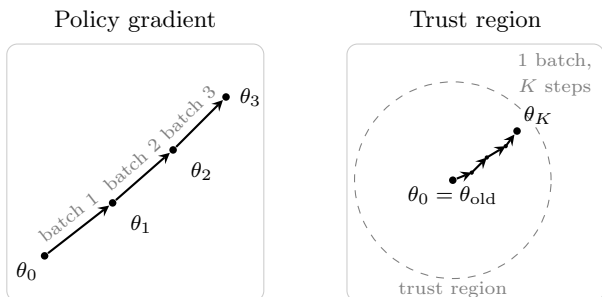


Figure 4.6: Two ways of using rollout data, in parameter space. Policy gradient methods (left) take a single step per batch of trajectories and must re-collect with the new policy to continue. Trust region methods (right) take K steps on a single batch, staying within the region where the surrogate $L_{\theta_{\text{old}}}$ remains trustworthy.

Thus, to make π_θ better than $\pi_{\theta_{\text{old}}}$, we want π_θ to put mass on actions with positive advantage *under* $\pi_{\theta_{\text{old}}}$.

This identity is exact but not what we want yet: the expectation is over the visitation distribution of the new policy, which we cannot sample from without rolling out π_θ in the environment, and the whole point is to choose θ *before* a new rollout. Hence, we will build a surrogate objective $L_{\theta_{\text{old}}}(\theta) \approx J(\theta) - J(\theta_{\text{old}})$.

First step: state distribution approximation. We simply swap d^{π_θ} for $d^{\pi_{\theta_{\text{old}}}}$:

$$L_{\theta_{\text{old}}}(\theta) := \frac{1}{1 - \gamma} \mathbb{E}_{s \sim d^{\pi_{\theta_{\text{old}}}}, a \sim \pi_\theta} [A^{\pi_{\theta_{\text{old}}}}(s, a)].$$

The approximation error of this surrogate grows with how much π_θ visits different states than $\pi_{\theta_{\text{old}}}$. This approximation thus dictates the existence of a **trust region** around $\pi_{\theta_{\text{old}}}$.

Second step: importance sampling. The expectation still requires sampling actions from π_θ . We rewrite it using importance sampling against $\pi_{\theta_{\text{old}}}$:

$$L_{\theta_{\text{old}}}(\theta) = \frac{1}{1 - \gamma} \mathbb{E}_{\substack{s \sim d^{\pi_{\theta_{\text{old}}} \\ a \sim \pi_{\theta_{\text{old}}}}} \left[\frac{\pi_\theta(a \mid s)}{\pi_{\theta_{\text{old}}}(a \mid s)} A^{\pi_{\theta_{\text{old}}}}(s, a) \right].$$

Reminder: importance sampling. To estimate an expectation under a distribution p when we can only sample from another distribution q , we use the identity

$$\mathbb{E}_{x \sim p}[f(x)] = \mathbb{E}_{x \sim q} \left[\frac{p(x)}{q(x)} f(x) \right],$$

valid as long as $q(x) > 0$ wherever $p(x) > 0$. The ratio $p(x)/q(x)$ corrects the bias introduced by sampling from q rather than from p .

Now everything is estimable from a batch of trajectories collected by $\pi_{\theta_{\text{old}}}$.

We can trust $L_{\theta_{\text{old}}}(\theta)$ only when π_θ is close to $\pi_{\theta_{\text{old}}}$. Indeed, if we naively maximize $L_{\theta_{\text{old}}}(\theta)$ to convergence, we will likely land on a θ for which the swap $d^{\pi_\theta} \rightarrow d^{\pi_{\theta_{\text{old}}}}$ is inaccurate, and for which the actual return $J(\theta)$ may even drop.

This gives the constrained problem:

$$\max_{\theta} L_{\theta_{\text{old}}}(\theta) \quad \text{s.t.} \quad \pi_\theta \text{ is close to } \pi_{\theta_{\text{old}}}.$$

i.e.

$$\max_{\theta} \mathbb{E}_{\substack{s \sim d^{\pi_{\theta_{\text{old}}}} \\ a \sim \pi_{\theta_{\text{old}}}}} \left[\frac{\pi_\theta(a|s)}{\pi_{\theta_{\text{old}}}(a|s)} A^{\pi_{\theta_{\text{old}}}}(s, a) \right] \quad \text{s.t.} \quad \pi_\theta \approx \pi_{\theta_{\text{old}}}$$

Different ways to enforce the closeness give different algorithms. **TRPO** [Schulman, Levine, et al. 2015] uses an explicit KL constraint

$$\mathbb{E}_{s \sim d^{\pi_{\theta_{\text{old}}}}} [D_{\text{KL}}(\pi_{\theta_{\text{old}}} \parallel \pi_\theta)] \leq \delta,$$

solved with a conjugate-gradient step on a quadratic approximation of the KL. It works but is heavy to implement and to scale. **PPO** [Schulman, Wolski, et al. 2017] replaces this constraint with a much simpler proxy.

4.3.2 PPO

PPO modifies the previous objective as follows:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_{\pi_{\theta_{\text{old}}}} \left[\min \left(\rho_t(\theta) \hat{A}_t, \text{clip}(\rho_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right]$$

where $\rho_t(\theta) = \frac{\pi_{\theta}(A_t | S_t)}{\pi_{\theta_{\text{old}}}(A_t | S_t)}$ is the **importance ratio**, $\hat{A}_t$ is an advantage estimator (typically GAE, see Section 4.2.4), and ϵ is a small constant (commonly 0.1 or 0.2) called the **clip parameter**.

The intuition is to limit how far each action can be pushed in the direction of the objective:

- If $\hat{A}_t > 0$, we want to increase $\pi_{\theta}(A_t | S_t)$, so ρ_t tends to grow. The clip caps the gain at $\rho_t = 1 + \epsilon$: past that, the gradient is zero.
- If $\hat{A}_t < 0$, we want to decrease $\pi_{\theta}(A_t | S_t)$, so ρ_t tends to shrink. The clip caps the loss at $\rho_t = 1 - \epsilon$.
- In both cases, when ρ_t drifts outside the trust region against the sign of $\hat{A}_t$ (e.g. $\rho_t < 1 - \epsilon$ with $\hat{A}_t > 0$), the min selects the unclipped term so the gradient remains non-zero and pulls the policy back.

PPO thus implements a soft trust region: the policy is softly prevented from moving too far on any single sample.

PPO performs several gradient steps (typically

around 5 epochs) on the same batch, re-using the same data each time. As updates accumulate, ratios start hitting the clip and the gradient on those samples gets zeroed, which slows down the drift between θ and θ_{old} . The rest of the algorithm structure is the same as VPG with GAE:

Algorithm – PPO

```

1: Init policy parameters  $\theta$  and critic parameters  $\phi$ 
2: for iteration = 1, ...,  $M$  do
3:   Collect a batch of  $N$  trajectories  $\{\tau_i\}_{i=1}^N$  with  $\pi_\theta$ 
4:   for each trajectory  $\tau_i$  do
5:     Compute  $\forall t \hat{A}_t^{(i)}$  via GAE using  $V_\phi$ 
6:   end for
7:   Set  $\theta_{\text{old}} \leftarrow \theta$  and cache  $\log \pi_{\theta_{\text{old}}}(a_t^{(i)} | s_t^{(i)})$ 
8:   for epoch = 1, ...,  $K$  do
9:     for each minibatch  $\mathcal{B}$  of size  $B$  do
10:       $\rho_t^{(i)} \leftarrow \exp(\log \pi_\theta(a_t^{(i)} | s_t^{(i)}) - \log \pi_{\theta_{\text{old}}}(a_t^{(i)} | s_t^{(i)}))$ 
11:      Policy update:
12:       $\theta \leftarrow \theta + \alpha \nabla_\theta \frac{1}{|\mathcal{B}|} \sum_{(i,t) \in \mathcal{B}} \min(\rho_t^{(i)} \hat{A}_t^{(i)}, \text{clip}(\rho_t^{(i)}, 1 - \epsilon, 1 + \epsilon) \hat{A}_t^{(i)})$ 
13:      Critic update:
14:       $\phi \leftarrow \phi - \beta \nabla_\phi \frac{1}{|\mathcal{B}|} \sum_{(i,t) \in \mathcal{B}} \frac{1}{2} (\text{sg}(\hat{A}_t^{(i)} + V_\phi(s_t^{(i)})) - V_\phi(s_t^{(i)}))^2$ 
15:    end for
16:  end for
17: end for

```

Code: [little-book-rl/algos/policy_based/ppo.py](https://github.com/openai/little-book-of-rl/blob/master/chapters/policy_based_ppo.py)

Remark

It is worth observing that the very first gradient step of PPO (for which $\rho_t = 1$ everywhere) recovers the policy gradient with advantages of VPG from §4.2.3, even though the derivation went through the performance difference lemma rather than the policy gradient theorem.

Also, while we can re-use the same batch of data collected by a previous policy to update the current one, we still mostly refer to PPO as an on-policy method, since the surrogate objective is only valid for policies that are close enough to the one that produced the data.

Finally, the attentive reader may have noticed that the number of gradient steps performed per PPO iteration is not K but $K \times \text{num_minibatches}$: at each epoch, the collected batch is shuffled and split into minibatches, and one gradient step is taken on each. This allows for even more updates per batch, with the clipping ratio making it possible.

Remark

There are many implementation details specific to a given PPO codebase (or any deep RL algorithm) that we have intentionally omitted. A few worth noting:

- *Defining a collection period.* Should we collect N full episodes or T timesteps before each update? Fixing the number of episodes is closer to the textbook definition, but the wall-clock cost per update fluctuates as episode lengths shift with the policy. Fixing T timesteps is more common as it gives a constant compute budget per iteration and works more naturally with vectorized environments, at the price of having to bootstrap $V(s_T)$ on the last (truncated) step.
- *Termination vs. truncation.* A terminal state has $V(s_T) = 0$, whereas a time-limit cutoff requires bootstrapping with the critic’s estimate. Some implementations treat truncations as terminations.

The PPO implementation in the supplementary material of the book lists the differences between the pseudocode shown here and the PPO of CLEANRL. Also, see Huang et al. (2022), *The 37 Implementation Details of PPO*, for a thorough catalogue.

Example: Atari Breakout

Similarly to DQN, we apply PPO to the Atari Breakout environment. It yields:

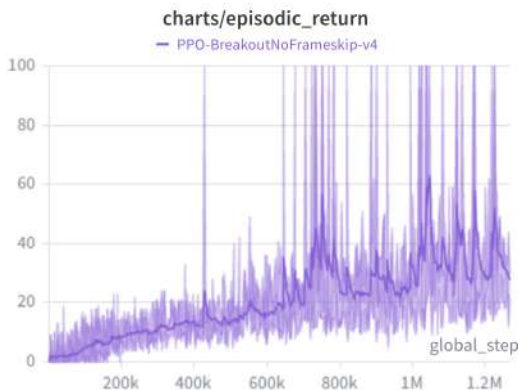


Figure 4.7: Rewards obtained by PPO on the Atari Breakout environment.

Monitoring the policy loss gives little insight:



Figure 4.8: Policy loss during training.

but the entropy of the policy is more interesting as it shows how deterministic the policy becomes (and thus if it is still evolving or not):



Figure 4.9: Entropy of the policy during training.

4.3.3 VPG vs. PPO

On CartPole, we compare how robust VPG and PPO are to re-using the same batch of trajectories for multiple gradient steps. An algorithm capable of doing so extracts more information from each batch, and thus is more data-efficient.

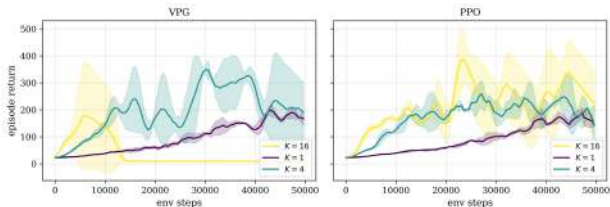


Figure 4.10: Episode return as a function of total training environment steps on CartPole, for VPG (left) and PPO (right) with $K \in \{1, 4, 16\}$ epochs per batch. Because each batch is split into 8 mini-batches, each epoch corresponds to 8 gradient steps.

The setting $K = 1$ gives very similar performance for both because, as we noted earlier, the first gradient step of PPO is the same as VPG. However, while raising K from 1 to 16 yields faster convergence for PPO, it is much more brittle and even leads to irrecoverable policy collapse for VPG. This is explained by the following figure:

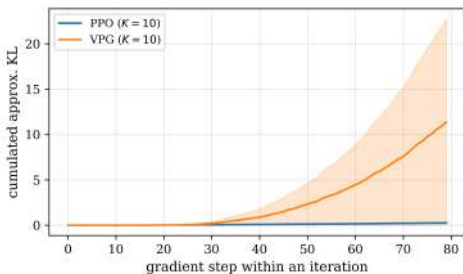


Figure 4.11: Cumulative approximate KL divergence between the current policy and the data-generating policy, measured per mini-batch gradient step within a single iteration ($K = 10$ epochs $\times$ 8 mini-batches = 80 steps)

Put differently, PPO makes sure that the new policy behaves similarly to the previous one. One could argue that using a smaller step size for VPG would also make it possible to re-use the same batch, but this would control the drift in parameter space rather than in policy space. The approximation of the surrogate objective is done with $d^{\pi_{\theta}} \approx d^{\pi_{\theta_{\text{old}}}}$, whose validity requires closeness in policy space, which is not necessarily obtained by closeness in parameter space.

PPO limits the drift in policy space by stopping the gradient on actions which have already drifted too far, while still making learning progress on the rest of the batch.

Part III

RL at scale

Chapter 5

RL $\times$ LLMs

So far we have built up the RL toolkit on general problems. This chapter applies that toolkit to a specific setting: LLM training.

In this setting, the Large Language Model (LLM) that we want to train has already been *pretrained* on trillions of tokens and is a very powerful conditional distribution over text, that we refer to as the base model. At that stage, it encodes a lot of knowledge but its behaviour is very hard to predict and control, making that knowledge difficult to leverage.

This is why one of the central efforts in the LLM field since late 2022 has been fine-tuning base

models. The most direct option is through *supervised fine-tuning* (SFT), which consists of collecting demonstrations of the desired behaviour and training the model to imitate them. While being simple to implement and somewhat effective, it requires demonstrations. RL on the other hand needs no demonstrations, only a reward function that scores model outputs and lets the model search for behaviour that scores well.

This can be preferable to SFT because RL can learn behaviour that was never demonstrated such as decomposing a problem, considering alternatives or checking its own work. Also, because the signal comes from a reward rather than from a fixed human-written target, RL can in principle push models past the level of any specific human demonstrator.

Two distinct generations of RL training of LLMs have emerged so far.

First generation: alignment (RLHF). The first wave, around 2022, used RL to turn a base model into a chatbot or assistant. It consisted first in training a reward model able to predict human preferences between two model outputs, and then using that reward to fine-tune the base model with RL, i.e., reinforcing the generated tokens that led to highly-rewarded sequences. This

RLHF (Reinforcement Learning with Human Feedback) recipe was very effective and behind the success of products like ChatGPT. However, it turned out that RL on its own was not strictly necessary to such alignment procedures. The RL was constrained both by how reliable the reward model was and by what it actually rewarded. Predicting human preferences is quite a shallow signal determined mostly by tone and style. It gives the model no incentive to reason, plan, or develop any new capability.

Second generation: reasoning (RLVR).

The second wave, starting in 2024, uses RL to train models in environments with verifiable rewards (**Reinforcement Learning with Verifiable Rewards**, RLVR). The most common setup is to give the model math problems and reward it for producing the correct numerical answer. A byproduct of this training setup is that it teaches the model to reason and learn *thinking actions* like planning, decomposing a problem, backtracking or self-checking. Such verifiable rewards are natural to obtain on STEM tasks, where correctness can be checked mechanically, but much harder to construct for open-ended tasks like creative writing.

This setup differs from RLHF in that the reward

is more “interesting” to optimize against (i.e., optimising against it tends to develop or resurface reasoning capabilities) and is more robust to reward hacking. This makes it practical to run RLVR at much larger scale than RLHF, often approaching a non-trivial fraction of pretraining ($>10\%$).

The rest of this chapter focuses on RLVR.

5.1 Setup

A prompt x is sampled from a dataset $\mathcal{D}$. The model generates a response $y = (y_1, \dots, y_T)$ one token at a time, until it emits an end-of-sequence token or hits a maximum length.

- the **state** at step t is the prompt concatenated with the tokens generated so far, $S_t = (x, y_{<t})$.
- the **action** is the next token $A_t = y_t \in \mathcal{V}$, drawn from the vocabulary.
- in the RLVR setting, the **reward** is sparse and terminal: a verifier scores the full response and returns $r(x, y) \in \mathbb{R}$.
- the **policy** $\pi_\theta(A_t \mid S_t)$ is the LLM itself: it gives a distribution over the actions (i.e. the next token to generate).

- **transitions** are deterministic concatenation, $S_{t+1} = (S_t, A_t)$.

We keep a reference policy π_{ref} , typically the SFT model we start from. Most LLM RL objectives include a KL penalty $\beta \cdot D_{\text{KL}}(\pi_{\theta} \parallel \pi_{\text{ref}})$ that keeps the trained policy close to it. This regularizer prevents collapse to degenerate token distributions that exploit the reward without producing readable text.

5.2 From PPO to GRPO

The action space being large and the rewards sparse with long horizons, methods like DQN are typically not a good fit, hence policy optimization methods are used, and in particular trust region methods.

Applying PPO to the previous setup gives:

$$J^{\text{PPO}}(\theta) = \mathbb{E}_{\substack{x \sim \mathcal{D} \\ y \sim \pi_{\theta_{\text{old}}}}} \left[\sum_{t=1}^{|y|} \min \left(\rho_t(\theta) \hat{A}_t, \text{clip}(\rho_t(\theta), 1-\epsilon, 1+\epsilon) \hat{A}_t \right) \right]$$

where $\rho_t(\theta) = \frac{\pi_{\theta}(y_t|x, y_{<t})}{\pi_{\theta_{\text{old}}}(y_t|x, y_{<t})}$ is the per-token importance ratio and $\hat{A}_t$ is the GAE advantage computed from a learned value function V_{ϕ} typically as large as the policy. Such critic is expensive in

memory and also hard to train since the reward is sparse and only available at the end of quite long trajectories.

GRPO (Group Relative Policy Optimization [Shao et al. 2024; DeepSeek-AI 2025]) drops the critic and replaces it by running the same prompt multiple times to get a group of outputs $\{y_1, \dots, y_G\}$. This group led to rewards $R = \{R_1, \dots, R_G\}$. Then, the advantage of each response is computed as $\hat{A}_{i,t} = R_i - \text{mean}(R)$.

Plugging this advantage into the PPO clipped surrogate and averaging over the group, and adding a per-token KL penalty against π_{ref} gives the GRPO objective:

$$J^{\text{GRPO}}(\theta) = \mathbb{E}_{\substack{x \sim \mathcal{D} \\ \{y_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot|x)}} \left[\frac{1}{G} \sum_{i=1}^G \sum_{t=1}^{|y_i|} \left(\min(\rho_{i,t}(\theta) \hat{A}_i, \text{clip}(\rho_{i,t}(\theta), 1-\epsilon, 1+\epsilon) \hat{A}_i) - \beta D_{\text{KL}}(\pi_{\theta} \parallel \pi_{\text{ref}}) \right) \right]$$

with $\rho_{i,t}(\theta) = \frac{\pi_{\theta}(y_{i,t}|x, y_{i,<t})}{\pi_{\theta_{\text{old}}}(y_{i,t}|x, y_{i,<t})}$. So GRPO is no longer an actor-critic method but still is a trust region method.

While GRPO was developed in the context of RL for LLMs, it is a general RL algorithm that can be applied to any environment with the requirements that (1) only terminal rewards are available and (2) we are able to start multiple rollouts from the same starting state.

5.3 Beyond GRPO

From GRPO, several observations have been made and improvements proposed.

New capabilities? Some works have shown that RLVR only elicits capabilities that were already present in the base model. Yue et al. (2025) shows setups where RL only improves pass@1 scores and not pass@8 scores, it shows that RL just learns to better sample an answer that already exists in the model. This is not a consensus at all, as later work has shown that with prolonged RL training and a diverse task mix, models do surpass their base counterpart on pass@ k at large k [Liu et al. 2025; Cursor Research 2026].

High-entropy tokens RLVR training mostly impacts high-entropy tokens (tokens for which the model is uncertain) [Wang et al. 2025]. These

usually represent forking tokens in the reasoning path like “therefore”, “perhaps” or “however”.



Figure 5.1: Tokens generated from a high-entropy distribution. Extracted from Figure 2 of Wang et al. (2025).

Different strategies have been proposed to further boost the impact of RL on these tokens. Most consist in raising the upper clipping ratio to allow for more aggressive updates on these tokens (DAPO, JUSTRL, Lite-PPO, MiniRL). Others limit the update on these tokens. These strategies also help slow the entropy decrease of the policy.

Length normalization The original GRPO objective weights down each trajectory gradient by its length ($\frac{1}{|y_i|}$), thereby weighting every token equally between the trajectories. For some (DAPO, Dr. GRPO), this is an error carried from supervised learning and has no reason to be here as it biases answers toward shorter positive

trajectories or long negative ones. Others argue that it is a useful bias to have to promote long answers and thus encourage reasoning chains to emerge.

Reward shaping An active line of work introduces explicit length-related reward terms, motivated by the observation that reasoning models tend to over-expand their CoT even on trivial prompts [Chen et al. 2025]. The simplest schemes apply a constant penalty on the length of the response. Applying such penalty too early or too strongly tends to collapse exploration and accuracy. More refined schemes like Cursor Composer 2 adapt the length penalty to the task difficulty, so the model is forced to be concise on easy prompts but can think for longer when the problem warrants it.

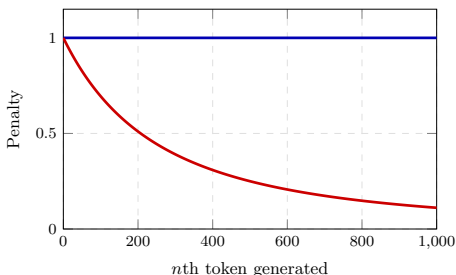


Figure 5.2: Penalty incurred by generating the n th token of a rollout. Blue is constant penalty, red is Composer 2.

Dynamic sampling Some setups (DAPO, INTELLECT-3) ensure that training prompts produce a useful learning signal. This can be done within a batch (filtering out prompts whose rollouts are all correct or all incorrect), or over a longer horizon (curriculum-style sampling that progressively favors harder prompts).

Sequence-level objective As only one reward is obtained at the very end of the episode, the MDP can be reformulated at a sequence-level:

$$J^{\text{GRPO,seq}}(\theta) = \mathbb{E}_{\substack{x \sim \mathcal{D} \\ \{y_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot|x)}} \left[\frac{1}{G} \sum_{i=1}^G \min(s_i(\theta) \hat{A}_i, \text{clip}(s_i(\theta), 1-\epsilon, 1+\epsilon) \hat{A}_i) \right]$$

i.e. instead of considering each token generated as an action, we consider the generation of the whole sequence as a single action. A trajectory is thus made of a single transition, with a single state x (the prompt) and a single action (the whole response, y) and the reward. The importance ratio is then:

$$s_i(\theta) = \frac{\pi_{\theta}(y_i | x)}{\pi_{\theta_{\text{old}}}(y_i | x)} = \prod_t \rho_{i,t}(\theta)$$

It is a valid choice to make but is not made as is in practice, because the sequence-level impor-

tance ratio has variance that grows multiplicatively with $|y_i|$, which is prone to instabilities and makes the clipping range $[1-\epsilon, 1+\epsilon]$ hard to calibrate across rollouts of different lengths. From there:

- Zheng et al. (2025) shows that using a token-level objective is in fact an approximation of the sequence-level objective that is valid when $\theta \approx \theta_{\text{old}}$. They argue that the sequence-level objective is more principled for RLVR. Indeed for example, under the sequence-level objective, the (initial) state distribution is the same for any policy because it only depends on the dataset, hence the underlying trust-region approximation $d^{\pi_\theta} \approx d^{\pi_{\theta_{\text{old}}}}$ becomes exact rather than approximate.
- GSPO (and maybe DeepSeek-R1) optimizes the sequence-level objective but replaces the importance ratio by a length-normalized version to mitigate the variance issue mentioned above:

$$s_i(\theta) = \left(\frac{\pi_\theta(y_i | x)}{\pi_{\theta_{\text{old}}}(y_i | x)} \right)^{1/|y_i|}$$

5.4 RL Infrastructure

While the pseudocode of GRPO fits in fewer than 30 lines, implementing an efficient RLVR training loop is a substantial systems engineering effort [Team Olmo et al. 2025; Prime Intellect Team et al. 2025].

It relies on two main components:

- the **trainer**, which consumes batches of experience and runs GRPO (or other) to update the policy weights.
- the **inference engine**, which collects rollouts by executing the policy in the environment. It can involve tool use (like web requests) or code execution.

Each typically runs on multiple interconnected GPU nodes. The process thus mixes two very different workloads: training (compute-bound) and inference (memory bandwidth-bound), each with a different framework (Megatron or TorchTitan vs. vLLM or SGLang). The two must communicate and be orchestrated to minimize idle time on either side. Another difficulty is that as training progresses, the policy tends to generate longer rollouts (longer reasoning chains, more tool calls, ...), which makes inference disproportionately more expensive due to the quadratic cost of at-

tention in sequence length.

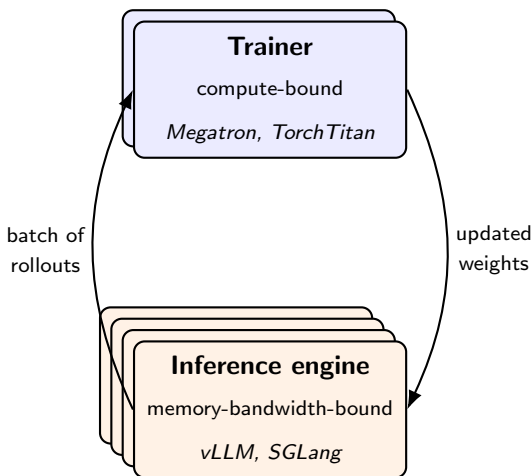


Figure 5.3: The two main components of an RLVR training loop. Each side is itself a fleet of GPU nodes, the inference fleet being allocated several times more GPU than the trainer (e.g. 3:1 ratio).

Several optimizations can be implemented to improve overall hardware utilization:

- **Asynchronous training and inference.**

The two components run in parallel without waiting for each other. Typically, after having collected a batch of rollouts, the inference engine doesn't wait for updated

weights from this batch to start collecting the next one. This makes the rollouts collected with an older policy and thus introduce off-policy-ness.

- **Continuous batching.** There is a lot of discrepancy between different rollouts in a same batch (response length, tool use, ...) and thus a lot of time can be lost waiting for the longest rollout to finish before starting the next batch. Continuous batching repopulates a rollout slot as soon as it is finished.
- **In-Flight weight updates.** This technique makes the inference engine continuously poll the trainer for new weights, thus using the latest policy as soon as it is available. This leads to a single rollout being generated with multiple policies, but in a sense limits off-policy-ness.

These optimizations are a trade-off between hardware efficiency and algorithmic fidelity, more particularly off-policy-ness.

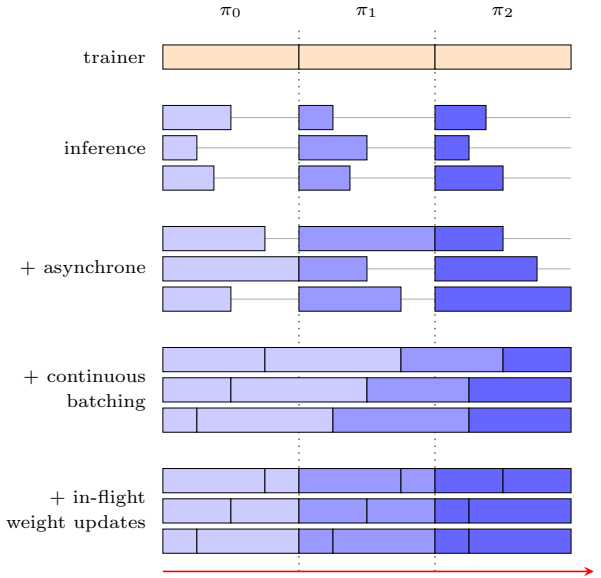


Figure 5.4: Overview of the different optimizations to improve hardware utilization in an RLVR training loop. The blue color intensity indicates the policy used to generate each rollout segment.

Another consideration to take into account is the mismatch between the training and inference engines. These are exacerbated by the presence of Mixture Of Experts (MoE) which can, from a slight numerical difference, route tokens to different experts which further increase the training-

inference mismatch. The rollout policy, denoted as μ_θ , while using the same weights as the training policy π_θ , thus produces different outputs which biases the gradient estimator. One fix [F. Yao et al. 2025] is to take this discrepancy into account in another importance sampling ratio:

$$\frac{\pi_{\theta_{\text{old}}}(y_t|x, y_{<t})}{\mu_{\theta_{\text{old}}}(y_t|x, y_{<t})}$$

which leads to the global importance sampling ratio:

$$\rho_t(\theta) = \frac{\pi_\theta(y_t|x, y_{<t})}{\mu_{\theta_{\text{old}}}(y_t|x, y_{<t})} = \frac{\pi_\theta(y_t|x, y_{<t})}{\pi_{\theta_{\text{old}}}(y_t|x, y_{<t})} \cdot \frac{\pi_{\theta_{\text{old}}}(y_t|x, y_{<t})}{\mu_{\theta_{\text{old}}}(y_t|x, y_{<t})}$$

this ratio corrects the **mismatch in action selection**: the first ratio corrects for the fact that we perform multiple updates and have therefore drifted from the old policy (derived in Section 4.3.2), while the second corrects the training-inference mismatch. The expectation is now under the rollout policy $\mu_{\theta_{\text{old}}}$, and we correct with $\rho_t(\theta)$.

5.5 Blurring the line between SFT and RL

In parallel, some methods propose to blur the line between SFT and RL in order to leverage the strength of both approaches. Indeed, one of

the main advantages of RL is that it performs on-policy training, i.e. it learns from situations that it actually encounters, rather than from situations encountered by an expert or a teacher. This is much more robust (as the student is likely to drift from the expert’s situations at inference) and enables *grounding* [Carta et al. 2023]. But RL provides a sparse learning signal. On the other hand, SFT provides a dense learning signal but performs off-policy training.

On-Policy Distillation [Lu et al. 2025], which can be viewed as a token-level version of **DAGger** [Ross et al. 2011] applied in the LLM context, samples situations from a student model and uses a teacher model to provide targets on these situations. It thus combines the advantages of on-policy training with a dense learning signal.

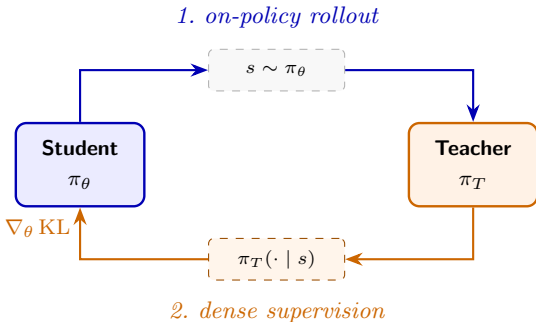


Figure 5.5: On-Policy Distillation.

Multiple works have shown that SFT tends to induce memorization and RL generalization [Chu et al. 2025; Shenfeld et al. 2025]. **DFT** (Dynamic Fine-Tuning, [Wu et al. 2026]) may offer an explanation by rewriting the SFT objective as a policy gradient objective:

$$\nabla_{\theta} \mathcal{J}^{\text{SFT}}(\theta) = \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_{\theta}(\cdot | x)} \left[\begin{aligned} & w(y | x) \nabla_{\theta} \log \pi_{\theta}(y | x) r(x, y) \end{aligned} \right]$$

with:

$$w(y | x) = \frac{1}{\pi_{\theta}(y | x)}, \quad r(x, y) = \mathbb{I}\{y = y^*\}$$

So SFT can be seen as policy gradient with a reward that is 1 if the model output matches the target and 0 otherwise weighted by a scalar that inflates updates on low-probability targets and biases the model toward memorizing specific (x, y^*) pairs instead of learning a generalizable policy.

Chapter 6

AlphaGo Zero

This chapter aims at giving a high-level overview of the famous AlphaGo Zero algorithm under the prism of the concepts we have built throughout the book.

You will find the ingredients of AlphaGo Zero familiar but assembled in a quite different way from what we have seen in the algorithms of either the value-based or policy-based families.

6.1 The main idea

AlphaGo Zero [Silver et al. 2017] can be seen as a form of *policy iteration* coupled with a model-

based approach. During training, it alternates between the two phases:

- **Improvement:** instead of acting according to the current policy, act according to an improved version of it. This improvement is done through planning by searching in the environment and thus leverages the known dynamics of the environment (hard-coded rules of Go).
- **Evaluation:** fit the current policy and value function to the experience collected using the improved policy.

It relies on neural networks to represent both the policy π_θ and the value function V_ϕ .

A particularly striking aspect of AlphaGo Zero is that training is done through *self-play*: the agent plays against another version of itself. This means that no human data is needed to train the agent, and that it can learn from scratch by playing against itself.

6.2 Improvement

The improvement step of AlphaGo Zero is done each time the agent has to pick an action, rather than globally at the end of an episode like in traditional policy iteration.

Considering state s_0 that the agent has to act on, the goal is to find an action a according to an improved version of the current policy π_θ .

This is done by traversing the Bellman backup diagram starting from s_0 . Instead of traversing it exhaustively, AlphaGo Zero grows a partial tree, concentrating computation on the regions of the tree that look promising under the current π_θ and V_ϕ :

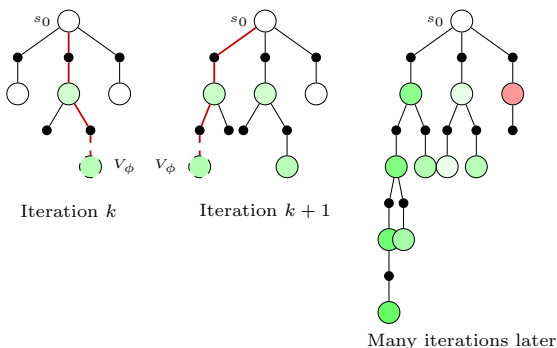


Figure 6.1: A single improvement step grows a partial tree through repeated traversals from s_0 . Each traversal (red) follows a selection rule trading off exploration and exploitation. It expands the tree by one node and backs up the statistics of the path it followed, starting from the bootstrap $V_\phi(\text{leaf})$. The most promising lines of play are thus the most densely explored. This is a modified version of an MCTS rollout.

After a fixed budget of traversals, the visit counts at the root concentrate on the most promising actions and define an improved policy at s_0 , from which the agent samples its action.

We can think about this improvement step as approximating the optimality Bellman operator [Grill et al. 2020]:

$$V_\phi(s_0) \leftarrow \max_a \mathbb{E}[r(s_0, a, s') + \gamma V_\phi(s')],$$

using the tree, and using the policy as a prior to focus the search and make it tractable.

We can also, more intuitively, think about the base policy π_θ as a kind of intuition policy, and the search as a “thinking” process that mentally unrolls and simulates the possible futures.

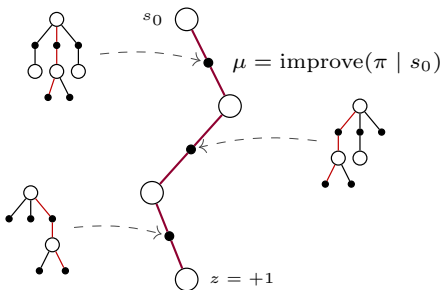


Figure 6.2: Each action of the result is the output of an inner search rooted at s_t . From one improvement step to the next, the tree can be partially reused.

6.3 Evaluation

Once experience has been collected by acting according to the improved policy and rewards z collected, the evaluation phase takes place.

The value function is fit to evaluate the improved policy, using MC evaluation:

$$\mathcal{L}_V(\phi) = \sum_s (V_\phi(s) - z)^2.$$

and the base policy is trained using behavior cloning / SFT to reproduce the improved policy:

$$\mathcal{L}_\pi(\theta) = - \sum_s \sum_a \mu(a|s) \log \pi_\theta(a|s),$$

The base policy π can be seen as the fast, intuitive System 1, and the search-improved policy μ as the slow, deliberate System 2. The evaluation phase distills System 2 back into System 1.

6.4 Self-play

Because Go is a two-player game, the agent needs an opponent to play against to collect experience. AlphaGo Zero uses self-play by playing against another version of itself. It also has the benefit that the agent always faces an opponent of its own level, providing some sort of curriculum learning.

Note that this self-play does not degenerate into a suboptimal strategy:

- the targets are grounded rather than self-referential: the regression is done toward a search-improved version of the policy and toward real game outcomes, rather than toward the network’s own predictions.
- the two-player zero-sum perfect-information structure ensures any exploitable weakness on one side is punished by the symmetric opponent.

but rather tends toward the unique minimax equilibrium of the game:

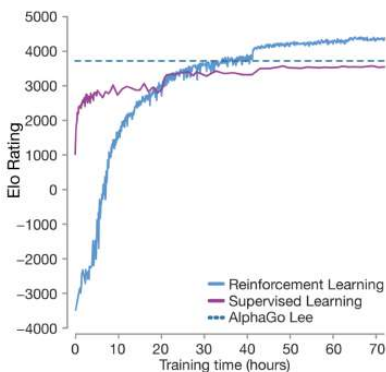


Figure 6.3: Elo rating of AlphaGo Zero during training. [Silver et al. 2017]

6.5 Discussion

At test time, AlphaGo Zero acts like it would in training, in the sense that it doesn't directly use its policy to pick actions but performs the MCTS-like improvement step described above to pick actions. This can be seen as a form of *test-time compute*.

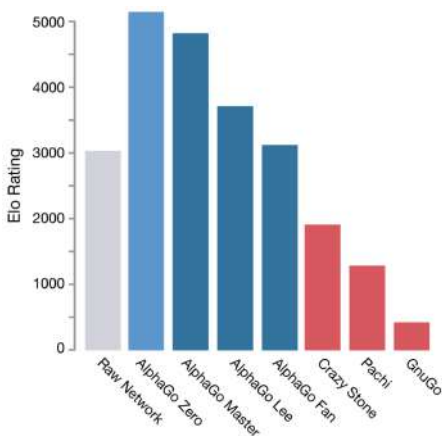


Figure 6.4: Using the base policy (“Raw Network”) is far less effective than using the search-improved policy (“AlphaGo Zero”), as it loses against its improved version more than 99.99% of the time. [Silver et al. 2017]

The AlphaGo Zero recipe, named AlphaZero, is thus very strong, but it relies on a specific setup:

known or learnable dynamics, a tractable action space, and cheap simulated rollouts.

It transfers remarkably well when these conditions hold (AlphaTensor, AlphaDev, AlphaProof, Aristotle), but it is far from a plug-and-play recipe for arbitrary settings. For example, there has been a strong push to apply it to LLM training with yet no clear public success so far [S. Yao et al. 2023; Hao et al. 2023].

Conclusion

Overview of the book

After some definitions (Chapter 1), we organized the methods by what they try to learn (Chapter 2). In the value-based family (Chapter 3), we saw different ways of traversing the same Bellman backup diagram, parameterized by two questions: *how deep* and *how wide*. In the policy-based family (Chapter 4), we instead tried to estimate the policy gradient with low variance, and how to safely take more than one step per batch of experience.

Finally, we looked at two popular instances of RL methods (Chapter 5, Chapter 6), and tried to link them to the concepts we built in the previous chapters.

Recurring patterns

To end the book, it is worth mentioning a few patterns that surfaced more than once across chapters.

Credit assignment. Every RL algorithm provides a way of attributing credit through time: bootstrapping in TD, returns in MC, the policy gradient theorem in policy optimization.

Bias and variance. They appear in the choice between MC and TD, in the depth of n -step returns, in the λ of GAE, in the importance ratio of PPO, in the group baseline of GRPO. Every estimator in RL faces this trade-off.

The data distribution moves. This is what fundamentally separates RL from supervised learning. The agent generates its own training data, and any update changes which data it will see next. Design choices like trust regions manage this drift.

On-policy vs off-policy. How stale is the data relative to the current policy? Q-learning tolerates arbitrary staleness by decoupling the behavior policy from the target policy, while PPO and GRPO use trust regions. It implies practical limitations in modern RL infrastructures.

What this book does not cover

Many active areas of RL were left out in this book, including:

- **Offline RL:** learning a policy from a fixed dataset without further interaction (CQL, IQL, behavior cloning baselines).
- **Deterministic policy gradients:** DDPG, TD3, SAC, and the off-policy actor-critic line of work behind modern robotics.
- **Model-based RL:** Dreamer, MuZero, PlaNet, and learned world models for planning.
- **Exploration:** multi-armed bandits, intrinsic motivation (RND, ICM), Thompson sampling, posterior sampling ...
- **Multi-agent RL:** general-sum games, equilibrium concepts, opponent modeling.
- **Inverse RL:** recovering rewards from demonstrations.
- **Hierarchical RL:** options, sub-goals, temporal abstraction.

Going further

There are many resources to dive deeper into RL. Among the citations of this book, you can notably find:

- *Reinforcement Learning: An Introduction*, Sutton and Barto [Sutton et al. 2018], which is the canonical reference for the classical theory.
- *Spinning Up in Deep RL*, [Achiam 2018], which provides a curated introduction to deep RL with clean PyTorch implementations. This can be completed with the more recent CLEANRL [Huang et al. 2022], as well as PUFFERLIB [Suarez 2024], which uses highly optimized C-vectorized environments, enabling fast iteration on RL algorithms.

You can also check:

- RL lecture series [Silver 2015], [Levine 2023], and [Abbeel 2021].
- Lilian Weng’s blog [Weng 2018], a really great resource covering RL topics (and more) with deep dives into each topic.
- *The RLHF Book*, [Lambert 2024], a thorough reference on post-training of language models.



Bibliography

- Abbeel, P. (2021). *Foundations of Deep RL – 6-lecture series*. YouTube lecture series. Accessed: 2026-05-15.
- Achiam, J. (2018). “Spinning Up in Deep Reinforcement Learning”. In.
- Bellman, R. (1961). *Adaptive Control Processes: A Guided Tour*. Princeton, NJ: Princeton University Press.
- Carta, T., C. Romac, T. Wolf, S. Lamprier, O. Sigaud, and P.-Y. Oudeyer (2023). “Grounding Large Language Models in Interactive Environments with Online Reinforcement Learning”. In: *Proceedings of the 40th International Conference on Machine Learning (ICML)*. Vol. 202. Proceedings of Machine Learning Research. PMLR, pp. 3676–3713. arXiv: [2302.02662](#) [[cs.LG](#)].

- Chen, X., J. Xu, T. Liang, Z. He, J. Pang, D. Yu, L. Song, Q. Liu, M. Zhou, Z. Zhang, R. Wang, Z. Tu, H. Mi, and D. Yu (2025). *Do NOT Think That Much for $2+3=?$ On the Overthinking of o1-Like LLMs*. arXiv: [2412.21187 \[cs.CL\]](#).
- Chu, T., Y. Zhai, J. Yang, S. Tong, S. Xie, D. Schuurmans, Q. V. Le, S. Levine, and Y. Ma (2025). *SFT Memorizes, RL Generalizes: A Comparative Study of Foundation Model Post-training*. arXiv: [2501.17161 \[cs.LG\]](#).
- Cursor Research (2026). *Composer 2 Technical Report*. arXiv: [2603.24477 \[cs.SE\]](#).
- DeepSeek-AI (2025). *DeepSeek-V3.2: Pushing the Frontier of Open Large Language Models*. arXiv: [2512.02556 \[cs.CL\]](#).
- Fleuret, F. (2023). *The Little Book of Deep Learning*. lulu.com.
- Grill, J.-B., F. Alth  , Y. Tang, T. Hubert, M. Valko, I. Antonoglou, and R. Munos (2020). “Monte-Carlo Tree Search as Regularized Policy Optimization”. In: *Proceedings of the 37th International Conference on Machine Learning (ICML)*. Vol. 119. Proceedings of Machine Learning Research. PMLR, pp. 3769–3778. arXiv: [2007.12509 \[cs.LG\]](#).
- Hao, S., Y. Gu, H. Ma, J. J. Hong, Z. Wang, D. Z. Wang, and Z. Hu (2023). “Reasoning with Language Model is Planning with World

- Model”. In: *Proceedings of EMNLP*. eprint: [2305.14992](#).
- Huang, S., R. F. J. Dossa, C. Ye, J. Braga, D. Chakraborty, K. Mehta, and J. G. Araújo (2022). “CleanRL: High-quality Single-file Implementations of Deep Reinforcement Learning Algorithms”. In: *Journal of Machine Learning Research* 23.274, pp. 1–18.
- Lambert, N. (2024). *Reinforcement Learning from Human Feedback*. Online.
- Levine, S. (2023). *CS285: Deep Reinforcement Learning*. Course, University of California, Berkeley. Accessed: 2026-05-15.
- Liu, M., S. Diao, X. Lu, J. Hu, X. Dong, Y. Choi, J. Kautz, and Y. Dong (2025). *ProRL: Prolonged Reinforcement Learning Expands Reasoning Boundaries in Large Language Models*. arXiv: [2505.24864 \[cs.CL\]](#).
- Lu, K. and Thinking Machines Lab (Oct. 2025). *On-Policy Distillation*. DOI: [10.64434/tml.20251026](#). URL: <https://thinkingmachines.ai/blog/on-policy-distillation> (visited on 05/10/2026).
- Mnih, V., K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis (2015). “Human-level control

through deep reinforcement learning”. In: *Nature* 518.7540, pp. 529–533. DOI: [10.1038/nature14236](https://doi.org/10.1038/nature14236).

Prime Intellect Team, M. Senghaas, F. Obeid, S. Jaghouar, W. Brown, J. M. Ong, D. Auras, M. Sirovatka, J. Straube, A. Baker, S. Müller, J. Mattern, M. Basra, A. Ismail, D. Scherm, C. Miller, A. Patel, S. Kirsten, M. Sieg, C. Reetz, K. Erdem, V. Weisser, and J. Hagemann (2025). *INTELLECT-3: Technical Report*. Submitted 18 Dec 2025. arXiv: [2512.16144 \[cs.LG\]](https://arxiv.org/abs/2512.16144).

Ross, S., G. J. Gordon, and J. A. Bagnell (2011). “A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning”. In: *Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS)*. Vol. 15. Proceedings of Machine Learning Research. PMLR, pp. 627–635. arXiv: [1011.0686 \[cs.LG\]](https://arxiv.org/abs/1011.0686).

Schulman, J., S. Levine, P. Moritz, M. I. Jordan, and P. Abbeel (2015). “Trust Region Policy Optimization”. In: *Proceedings of the 32nd International Conference on Machine Learning (ICML)*. Vol. 37. Proceedings of Machine Learning Research. PMLR, pp. 1889–1897. arXiv: [1502.05477 \[cs.LG\]](https://arxiv.org/abs/1502.05477).

Schulman, J., P. Moritz, S. Levine, M. I. Jordan, and P. Abbeel (2016). “High-Dimensional Continuous Control Using Generalized Advantage

- Estimation”. In: *4th International Conference on Learning Representations (ICLR)*. arXiv: [1506.02438 \[cs.LG\]](#).
- Schulman, J., F. Wolski, P. Dhariwal, A. Radford, and O. Klimov (2017). *Proximal Policy Optimization Algorithms*. arXiv: [1707.06347 \[cs.LG\]](#).
- Shao, Z., P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. K. Li, Y. Wu, and D. Guo (2024). *DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models*. arXiv: [2402.03300 \[cs.CL\]](#).
- Shenfeld, I., J. Pari, and P. Agrawal (2025). *RL’s Razor: Why Online Reinforcement Learning Forgets Less*. arXiv: [2509.04259 \[cs.LG\]](#).
- Silver, D. (2015). *Reinforcement Learning*. Lecture series, University College London (COMPM050/COMPGI13). Accessed: 2026-05-15.
- Silver, D., J. Schrittwieser, K. Simonyan, I. Antonoglou, A. Huang, A. Guez, T. Hubert, L. Baker, M. Lai, A. Bolton, Y. Chen, T. Lillicrap, F. Hui, L. Sifre, G. van den Driessche, T. Graepel, and D. Hassabis (2017). “Mastering the game of Go without human knowledge”. In: *Nature* 550.7676, pp. 354–359. DOI: [10.1038/nature24270](#).
- Suarez, J. (2024). *PufferLib: Fast and Sane Simplifying Reinforcement Learning for Complex*

- Game Environments*. <https://github.com/PufferAI/PufferLib>. GitHub repository.
- Sutton, R. S. and A. G. Barto (2018). *Reinforcement Learning: An Introduction*. 2nd ed. Cambridge, MA: MIT Press.
- Team Olmo, A. Ettinger, A. Bertsch, B. Kuehl, D. Graham, D. Heineman, D. Groeneveld, F. Brahman, F. Timbers, H. Ivison, J. Morrison, J. Poznanski, K. Lo, L. Soldaini, M. Jordan, M. Chen, M. Noukhovitch, N. Lambert, P. Walsh, P. Dasigi, R. Berry, S. Malik, S. Shah, S. Geng, S. Arora, S. Gupta, T. Anderson, T. Xiao, T. Murray, T. Romero, and V. Graf (2025). *Olmo 3*. Submitted 15 Dec 2025, revised 14 Apr 2026. arXiv: [2512.13961](https://arxiv.org/abs/2512.13961) [cs.LG].
- Wang, S., L. Yu, C. Gao, C. Zheng, S. Liu, R. Lu, K. Dang, X. Chen, J. Yang, Z. Zhang, Y. Liu, A. Yang, A. Zhao, Y. Yue, S. Song, B. Yu, G. Huang, and J. Lin (2025). *Beyond the 80/20 Rule: High-Entropy Minority Tokens Drive Effective Reinforcement Learning for LLM Reasoning*. arXiv: [2506.01939](https://arxiv.org/abs/2506.01939) [cs.CL].
- Weng, L. (2018). “A (Long) Peek into Reinforcement Learning”. In: *lilianweng.github.io*.
- Williams, R. J. (1992). “Simple statistical gradient-following algorithms for connectionist reinforcement learning”. In: *Machine Learning* 8.3–4. Papier fondateur introduisant l’algorithme REINFORCE et les

- méthodes de policy gradient, pp. 229–256. DOI: [10.1007/BF00992696](https://doi.org/10.1007/BF00992696).
- Wu, Y., Y. Zhou, Z. Ziheng, Y. Peng, X. Ye, X. Hu, W. Zhu, L. Qi, M.-H. Yang, and X. Yang (2026). “On the Generalization of SFT: A Reinforcement Learning Perspective with Reward Rectification”. In: *The Fourteenth International Conference on Learning Representations (ICLR)*. arXiv: [2508.05629 \[cs.LG\]](https://arxiv.org/abs/2508.05629).
- Yao, F., L. Liu, D. Zhang, C. Dong, J. Shang, and J. Gao (Aug. 2025). *Your Efficient RL Framework Secretly Brings You Off-Policy RL Training*.
- Yao, S., D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, and K. Narasimhan (2023). “Tree of Thoughts: Deliberate Problem Solving with Large Language Models”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. eprint: [2305.10601](https://arxiv.org/abs/2305.10601).
- Yue, Y., Z. Chen, R. Lu, A. Zhao, Z. Wang, Y. Yue, S. Song, and G. Huang (2025). *Does Reinforcement Learning Really Incentivize Reasoning Capacity in LLMs Beyond the Base Model?* arXiv: [2504.13837 \[cs.LG\]](https://arxiv.org/abs/2504.13837).
- Zheng, C., K. Dang, B. Yu, M. Li, H. Jiang, J. Lin, Y. Liu, H. Lin, C. Wu, F. Hu, A. Yang, J. Zhou, and J. Lin (2025). *Stabilizing Reinforcement Learning with LLMs: Formulation and Practices*. arXiv: [2512.01374 \[cs.LG\]](https://arxiv.org/abs/2512.01374).



This book is licensed under the
Creative Commons
Attribution-ShareAlike 4.0
International License
(CC BY-SA 4.0).

The full license text is available at:

creativecommons.org/licenses/by-sa/4.0

© 2026 Alexandre Torres-Leguet